

Concretely Efficient Fluid MPC with Linear Communication

Yubo Zeng¹ , Kang Yang^{2, (✉)} , Dengguo Feng¹ , and Min Zhang¹ 

¹ Laboratory of Trusted Computing and Information Assurance, Institute of Software, Chinese Academy of Sciences, Beijing, China

`zengyubo21@mails.ucas.ac.cn`

`fengdg@263.net`

`zhangmin@iscas.ac.cn`

² State Key Laboratory of Cryptology, Beijing, China

`yangk@sklc.org`

Abstract. Traditional Secure Multi-Party Computation (MPC) requires parties to stay online through the whole computation, which compromises scalability when dealing with large-scale and complex tasks. The notion of fluid MPC, introduced by Choudhuri et al. (Crypto 2021), aims to address this challenge by presenting a dynamic participation model where parties have the flexibility to join and leave as needed. The best-known honest-majority MPC protocol by Bienstock et al. (Crypto 2023) in the fluid setting achieves linear communication complexity, but still incurs a substantially higher communication overhead than MPC in the classical setting.

In this paper, we present two concretely efficient fluid MPC protocols in the honest-majority setting. The first, *Velora*, is an unconditionally secure maximal-fluid MPC protocol, which achieves the lowest communication cost among maximally fluid MPC protocols by introducing a new approach of transferring the output sharings held by the current committee to the next committee. To eliminate the inherent overhead of *Velora*, we also propose a separation-generation approach for random double sharings and integrate it into the second protocol as *lon*. As a trade-off, *lon* relaxes the fluidity requirement to the *submaximal fluidity*, allowing an extra internal communication round for each committee. Both protocols *Velora* and *lon* enable us to extend the ATLAS technique from the classical setting to the fluid setting for further lowering communication overhead.

Compared to the best-known fluid MPC protocol, our protocols reduce the communication cost per multiplication gate by a factor of $5.4 \sim 7.5\times$ (resp., $20.7 \sim 28\times$) for semi-honest security (resp., malicious security). Compared to the state-of-the-art ATLAS protocol by Goyal et al. (Crypto 2021) in the classical setting, our semi-honest protocols only introduce a $1 \sim 1.5\times$ larger communication overhead for securely computing multiplication gates.

1 Introduction

Secure Multi-Party Computation (MPC) [29,30] allows a set of mutually distrusting parties to jointly evaluate an arbitrary function on their private inputs, while preserving the privacy of the inputs and guaranteeing the correctness of the outputs. Unconditionally secure MPC protocols enjoy not only the information-theoretic security but also a lightweight computational cost. Such protocols can only be constructed in the honest-majority setting (i.e., at most $t < n/2$ parties are corrupted). An unconditionally secure MPC protocol guarantees security in the presence of either a semi-honest adversary who faithfully follows the protocol, or a malicious adversary who can arbitrarily deviate from it. One key goal of designing unconditionally secure MPC protocols is to reduce the communication cost that is the efficiency bottleneck. In particular, a line of works [10,19,22,8,25,21] focus on reducing the communication cost of unconditionally secure MPC protocols in the classical setting.

A fixed set of parties, who must participate for the entire computation duration of an MPC protocol, may not be realistic, due to unstable networks, computer crashes, software bugs, and so on. Dynamic participation, where parties can join and leave the computation, is more realistic, especially for complex, large-scale tasks with extended runtime. For example, according to the MPC survey [17], the Privacy-Preserving Machine Learning (PPML) protocol (called Falcon) [28] with malicious security takes over 3 years and about 1012 TB of communication to train a VGG-16 model over the Tiny ImageNet dataset in the honest-majority setting with three parties.

To overcome the above limitation of classical MPC protocols, Choudhuri et al. [9] introduced a dynamic model called Fluid MPC, which enables a set of parties to participate in one committee and execute over a specific number of rounds. When the computation task is completed, the parties in the committee need to securely transfer the intermediary state of the computation to the next committee, which continues to execute the protocol. Therefore, while classical MPC can be made communication-free for linear gates, it seems to be inevitable to communicate for linear gates between two committees in the fluid setting. The notion of fluidity expresses the flexibility and is defined by the number of round which one committee needs to be involved in. In the maximal-fluid setting, each committee needs to be active in only one communication round, which provides the best flexibility and maximally cuts down the online duration. Most of fluid MPC protocols [9,26,4,14,5,12] are designed to satisfy the maximal fluidity. In this work, we first focus on the maximal fluidity and aim to minimize the communication cost of Fluid MPC, followed by an exploration of the submaximal fluidity (i.e., fluidity 2) to attain superior communication efficiency.

Although existing (maximally) fluid MPC protocols minimize the online time of parties and have the best flexibility, their communication cost is much higher than classical MPC. Compared to the state-of-the-art unconditional MPC protocol (called ATLAS) [21] in the classical setting, the best-known fluid MPC protocol with unconditional security [4] blows up the communication overhead per multiplication gate by a factor of $7.5\times$ (resp., $57\times$) in the presence of semi-

Setting	Protocol	Comm. per Mult	Comm. per A&R	Malicious overhead	Rounds
Classical	ATLAS-1	4	None	$1\times$	$2d_C + O(1)$
	ATLAS-2	4.5	None	$1\times$	$d_C + O(1)$
Fluid (maximal)	[9]	n	n	$3\times$	$d_C + O(1)$
	[4]	30	6	$7.6\times$ for Mult $6\times$ for A&R	$2d_C + O(1)$
	Velora-1	5.5	3	$2\times$ for Mult $2.8\times$ for A&R	$2d_C + O(1)$
	Velora-2	6.75	1.5	$2\times$ for Mult $4.7\times$ for A&R	$d_C + O(1)$
Fluid (submaximal)	lon-1	4	3	$2\times$ for Mult $2.3\times$ for A&R	$2d_C + O(1)$
	lon-2	5.5	1.5	$2\times$ for Mult $3.7\times$ for A&R	$d_C + O(1)$

Table 1: **Comparison of communication cost per party and rounds between our fluid MPC protocols and the state-of-the-art fluid MPC protocols with optimal corruption threshold and unconditional security.** Here, “Comm. per A&R” represents the communication cost of transferring a single sharing for each addition or relay gate from the current committee to the next committee, and “Comm. per Mult” represents the communication cost of evaluating each multiplication gate. “Malicious Overhead” denotes the communication overhead of strengthening an MPC protocol from the semi-honest security to the malicious security. We use n to denote the number of parties in each committee, and denote by d_C the circuit depth.

honest (resp., malicious) adversaries. Hence, a significant communication gap exists between the classical and fluid settings.

1.1 Our Contribution

In the *maximal fluidity* setting, we propose a “pack-then-unpack” approach for transferring degree- t Shamir sharings from the current committee C_i to the later committee C_{i+2} . Specifically, C_i does not directly transfer degree- t sharings into C_{i+1} , and instead packs a batch of degree- t sharings into a packed secret sharing (PSS) to C_{i+1} . Then, the committee C_{i+2} can unpack the PSS to recover the batch of degree- t sharings via Lagrange interpolation. Building upon the transfer technique, we can realize fluid multiplication by following the DN protocol [10] and incorporating an extra transfer of one random sharing $[r]_t$.

Notably, the additional transmission of $[r]_t$ is inherent under the *maximal fluidity* setting, leading to a constant overhead gap in multiplication gate evaluation relative to the classical framework. In an effort to eliminate this limitation, we shift our focus to the *submaximal fluidity* setting, while simultaneously proposing a separation-generation approach of random double sharings for evaluating multiplication gates. Specifically, for evaluating multiplication gates, it is crucial to avoid repeatedly transferring random sharings. We solve this problem by separately generating a pair of random double sharings held by two adjacent committees, which does not introduce extra communication.

Furthermore, we extend the state-of-the-art ATLAS approach [21] for computing multiplication gates to the fluid model. Note that our approaches are suitable for extending both ATLAS-1 with lower communication and ATLAS-2 with less rounds. As a result, we constructed four concretely efficient fluid MPC protocols with linear communication, denoted as **Velora-1**, **Velora-2**, **lon-1**, and **lon-2**. Our protocols satisfy unconditional security in the honest-majority setting, while achieving the optimal corruption threshold ($t = \lfloor (n-1)/2 \rfloor$). We primarily design four semi-honest fluid MPC protocols and then demonstrate how to compile them to ensure security in the presence of malicious adversaries by presenting an efficient verification protocol.

In Table 1, we compare **Velora-1**, **Velora-2**, **lon-1**, and **lon-2** with previous unconditionally secure maximal-fluid MPC protocols, in terms of communication cost and rounds. Compared to the best-known fluid MPC protocol [4], **Velora-1** and **lon-1** reduce the communication cost per multiplication gate by a factor of $5.4 \sim 7.5\times$ (resp., $20.7 \sim 28\times$) in the presence of semi-honest adversaries (resp., malicious adversaries), when all protocols achieve the same rounds. Compared to [4], **Velora-2** and **lon-2** obtain fewer improvements for communication cost, i.e., $4.4 \sim 5.4\times$ (resp., $16.9 \sim 20.7\times$) for semi-honest security (resp., malicious security), but reduce the communication rounds by a half. For evaluating addition or relay gates, our protocols also improve the communication cost by a factor of either $2 \sim 4\times$ (resp., either $4.2 \sim 6.5\times$) for all of **Velora-1**, **Velora-2**, **lon-1**, and **lon-2** in the presence of semi-honest adversaries (resp., malicious adversaries). Compared to the state-of-the-art classical MPC protocol (named as ATLAS) [21], our protocols **lon-1** and **Velora-2** require $1\times$ and $1.5\times$ more communication cost for evaluating multiplication gates, respectively, while achieving the same rounds.

1.2 Related Work

Fluid MPC. Choudhuri et al. [9] introduced the Fluid MPC model and presented a formalized and modular definition framework. In the fluid setting, different parties voluntarily form each committee to execute in fewer communication rounds and leave as desired. [9] also gave the first fluid protocol in the honest majority setting under *maximal fluidity*, where each committee is restricted to one round of communication, leading to a quadratic communication overhead $O(n^2)$ per gate. By allowing a global preprocessing phase composed of crypto-

graphic primitives such as vector oblivious linear evaluation (VOLE) and pseudo-random generator (PRG), Rachuri and Scholl [26] realize the dishonest majority setting for the fluid model. However, [26] remains the quadratic communication cost under *maximal fluidity*. Not until Bienstock et al. [4] constructed the first maximally-fluid protocol based on DN-style operations with linear communication overhead in both the honest majority setting and the dishonest majority setting with preprocessing, and further extended to perfect security with abort while tolerating up to one-third corruption in [5].

To achieve a stronger version of security in the fluid setting, the foundational work [12] introduced the Layered MPC as a simple variant of the Maximally-Fluid MPC, executing with an interaction pattern represented by a directed layered graph of width n , where each committee of n parties is placed at a layer. While [12] established the feasibility of Guaranteed Output Delivery (GOD) with $t < n/3$, it necessitates a communication complexity of $O(n^9)$ per multiplication gate. Subsequent research [14] extended GOD to the optimal threshold $t < n/2$, albeit at an even more exorbitant cost of $O(n^{18})$. These results demonstrate that ensuring GOD under fluid settings inherently requires significant communication overhead.

Recently, a notable work [13] has identified that fluid MPC with *maximal fluidity* may overkill the entire efficiency, and turn to relax the fluidity for pursuing concrete efficiency. By relaxing the *maximal fluidity* to fluidity 5, they demonstrated that it is possible to maintain GOD while achieving a significant reduction in communication complexity with a factor of $O(n^3)$. This shift highlights a promising and largely unexplored optimization landscape in the trade-off between fluidity and communication efficiency.

YOSO MPC. Another extensively researched dynamic MPC is the “You Only Speak Once” (YOSO) model, introduced by Gentry et al. [20], where each party is randomly allocated to an actual physical entity called *role* executing some part of tasks in one committee ("speak") and then exiting after only send a single message ("death"). Specifically, YOSO involves two different mechanisms as Role Assignment and Role Execution. The former aims to map each role to real-world computational entities, whereas the latter concentrates on protocol design for the roles. This provides the benefit that the adversary is limited to random corruption, and it also facilitates the modular design of efficient protocols independently. Here, a series of works [15,2,7,6,23,24,16,11,3] have contributed significantly to the advancement of YOSO MPC.

2 Preliminaries

2.1 Notation

In this paper, we evaluate an arithmetic circuit over a large enough field $\mathbb{F}$. We denote C_i to be the committee that executes in the i -th epoch. For the *submaximal fluidity* setting, we denote C_i^1 to be the first communication round, and C_i^2 to be the second round for transferring messages to C_{i+1} . In addition, H_{C_i}

and $Corr_{C_i}$ denote the index set of honest parties in C_i and the set of corrupted parties in C_i , respectively. We assume that there are $n = 2t + 1$ clients forming a set as $C_{clnt} = \{U_1, \dots, U_n\}$. For the sake of simplicity, we may assume that $|C_i| = n$ for all i .

Given a positive integer p , we use $[p]$ to denote the set $\{1, 2, \dots, p\}$. For any threshold sharing $[x]$, we use $[x]_d^{C_i}$ to be a sharing held by C_i of degree d . In this paper, we use the standard Shamir secret sharing scheme [27] and the packed secret sharing scheme [18] to realize the fluid MPC. For more details, we recommend the readers to refer to Appendix C. When no ambiguity arises, we omit the subscript and use $[x]$ instead. For any two parties/servers that need communication, we assume that there exists a secure channel between them.

2.2 Fluid Model

According to [9], we present a brief introduction to formalize the fluid model. This model proceeds in three consecutive phases: Input, Execution, and Output stages. It is implemented in the *client-server* model, where the clients only provide inputs to and receive outputs, outsourcing the main computation to the servers and enjoying the benefit of modular proofs (referring to Appendix B for more details). Concretely, different servers are organized into a sequence of committees, where the first committee receives the inputs from the clients in the Input stage, and the last committee returns the outputs to the clients in the Output stage.

During the Execution phase, each committee handles a small segment termed as an epoch, which is further split into two sub-phases as *computation* phase and *hand-off* phase. In the previous sub-phase, the committee performs part of the computation task and transfers the current intermediate state to the next committee in the subsequent sub-phase to preserve protocol continuity. In addition, a server is called "active" if it either computes or sends/receives messages. Intuitively, it will activate from the *hand-off* phase of the previous committee and terminate after the same phase of the current committee.

Fluidity. Fluidity, defined as the minimum commitment required from each server, exhibits the flexibility of the protocol. We say that an MPC protocol realizes *maximal fluidity* as long as there exists with only a single round of communication per epoch. More specifically, in the *maximal fluidity* setting, each committee computes non-interactively and transfers the current state to the next committee in the only communication round.

It is worth noting that the last committee can have multiple committees in the Output phase to complete the whole protocol, i.e., not restricted by *maximal fluidity*. Another point is that the identity of the subsequent committee should be known to the current committee in its *hand-off* phase. Following [9], we assume that there exists an external mechanism that enables the current committee to identify the next committee.

Committee formation. Here, our framework imposes no limitations on committee formation. Each server can voluntarily participate in any committee given available computational resources and is allowed to join multiple committees.

Corruption. We consider a *malicious R -adaptive adversary* introduced in [9] and also used in [26,4,5] in the honest majority setting. Briefly speaking, the adversary statically corrupts a minority of clients. For each committee C_i , the adversary adaptively corrupts a minority of parties in C_i after learning the identities of C_i . Details can refer to Appendix A.

Layered Circuit. To allow the standardized fluid model to evaluate MPC smoothly, we transform the circuit to the original protocol into a layered circuit such that the output of gates on a layer must be used as inputs to the gates on the next layer. This property of layered circuits perfectly aligns with the fluid model, where the input of the next committee is completely from on the output of the previous committee. [9] has positively demonstrated that doubling the circuit width can convert any arithmetic circuit into a layered circuit. For more details, we recommend the reader to [9].

Security Model To securely model fluid MPC, we formalize an ideal Functionality $\mathcal{F}_{\text{dabb}}$ to capture the idealized behavior in the fluid setting. The Functionality $\mathcal{F}_{\text{dabb}}$, shown in Figure 1 and parameterized by a large enough field $\mathbb{F}$, performs four basic operations: Input, Add, Mult, and Output. It also captures dynamic parties, only occurring in the *hand-off* phase, where parties can interact with each other. For simplicity, we use the command Next-Committee of [4] to illustrate committee variations.

2.3 Building Blocks

Some Functionalities are required in our construction. Unlike the classical setting, normal ideal functionalities need to be redefined in the fluid model. We only present a concise description here, with the formal description and proofs referred to in Appendix D. Let $\mathcal{F}_{\text{rand}}$ be an ideal Functionality that distributes a random degree- t sharing $[r]_t$ to all parties in some committee C_i . Let $\mathcal{F}_{\text{coin}}$ be an ideal Functionality that samples a random element r to all parties in some committee C_i . Let $\mathcal{F}_{\text{zero}}$ be an ideal Functionality that distributes a random zero sharing $[0]_{2t}$ to all parties in some committee C_i . Let $\mathcal{F}_{\text{semi-mult}}$ be an ideal Functionality that receives two input sharings $[x]_t$ and $[y]_t$ from some committee C_i , and then outputs $[z = x \cdot y]_t$ to the following committee C_{i+2} . At the same time, we provide corresponding versions of each ideal Functionality tailored for the *submaximal fluidity* setting.

3 Technical Overview

This section gives a brief overview of the starting points and techniques used in our paper. Let C_i be the i -th committee that carries out the computation in the i -th epoch. For simplicity, we may assume that all committees, including

Functionality $\mathcal{F}_{\text{dabb}}$

Parameter: A large enough field $\mathbb{F}$, a universe U of all potential parties, a set of clients $C_{\text{clnt}} \in U$, and a public label (id_x, x) agreed by all parties for every variable x used in the computation.

Init: On input (Init, C) from all $P_i \in C_{\text{clnt}}$, in which case P_i sends the same set $C \in U$. If the set is consistent, set $i = 1$ and $C_1 = C$ as the first committee. Send (Init, C_1) to S . Otherwise, abort.

Input: On input (Input, id_x, x) from some parties $P_j \in C_{\text{clnt}}$, where all other parties of C_{clnt} should also input (Input, id_x) . Then store (id_x, x) and send (Input, id_x) to S . Otherwise, abort.

Change-Committee: On input $(\text{Change-Committee}, C)$ from all parties of the current committee C_i , where every party $P_j \in C_i$ sends the same set $C \in U$. If the set is consistent, upgrade $i = i + 1$ and $C_i = C$ as the next committee. Send $(\text{Change-Committee}, C_i)$ to S . Otherwise, abort.

Add: On input $(\text{Add}, id_z, id_x, id_y)$ from all parties of the current committee C_i , compute $z = x + y$ and store (id_z, z) . Send $(\text{Add}, id_z, id_x, id_y)$ to S .

Mult: On input $(\text{Mult}, id_z, id_x, id_y)$ from all parties of the current committee C_i , compute $z = x \cdot y$ and store (id_z, z) . Send $(\text{Mult}, id_z, id_x, id_y)$ to S .

Output: On input $(\text{Output}, \{id_{z_i}\}_{i \in [m]})$ from all parties of the final committee together with the clients set $C_{\text{clnt}} \cup C_i$, where each label id_{z_i} of value z_i is valid and stored, then retrieve $\{id_{z_i}, z_i\}_{i \in [m]}$ and send $(\text{Output}, \{id_{z_i}, z_i\}_{i \in [m]})$ to S . Wait for a command from S , if it is **Deliver**, then send the output to all clients. Otherwise, abort.

Fig. 1: **Functionality for dynamic arithmetic black box.**

the client committee, contain exactly $n = 2t + 1$ parties. In our *submaximal fluidity setting*, we specify that C_i^1 represents the first internal communication round of C_i , and C_i^2 represents the second communication round to transfer the current state to C_{i+1}^1 . We use $[x]_d$ to denote a degree- d Shamir secret sharing of the secret x and $\llbracket \mathbf{x} \rrbracket_d$ to denote a packed secret sharing of the secret vector $\mathbf{x}$. Furthermore, $[x]_d^{C_i}$ represents a sharing $[x]_d$ held by C_i .

In the following, we first point out the inefficiency dilemma of existing protocols under the *maximal fluidity* setting. Next, we present an efficient protocol for transferring degree- t Shamir sharings under *maximal fluidity*, and smoothly deduce an efficient multiplicative protocol in line with the DN protocol in Section 3.2. We then identify the inherent limitations of this construction in *maximal fluidity* setting and propose a more communication-efficient solution by leveraging the flexibility of *submaximal fluidity*. Subsequently, we extend the above protocols to our primary results *Velora* and *Ion* in Section 3.3. Finally, we illustrate how to advance our protocol against malicious adversaries in the last section.

3.1 The Inefficiency Bottleneck under *Maximal Fluidity*

The costs of the reshare and multiplication protocols constitute the main components of the communication overhead. Designing an efficient reshare protocol is the most challenging part of the fluid model. In the *maximal fluidity* setting, [9] presents a naive solution: Given a Shamir sharing $[x]_t = (x_1, \dots, x_n)$, the current committee distributes $[x_1]_t, \dots, [x_n]_t$ to the next committee, then the next committee locally sets $[x]_t = \sum_i L_{i,t} \cdot [x_i]_t$ with some Lagrange coefficients $L_{i,t}$, which inevitably cause quadratic communication. While [26] allows for a global preprocessing phase such that every pair of parties who may wish to be involved in the computation holds a common seed s , the parties of two adjacent committees can locally obtain some correlated randomness by applying these seeds into a common PRG (pseudo random generator), where resharing can be reduced to the current committee only sending shares to the first party in the next committee. However, a preprocessing phase is generally disallowed in the honest majority setting. [4] raises the degree of sharing from t to $2t$ and transfers a sharing $[x]_{2t}$ by directly sending $[x]_{2t} + [0]_{2t}$ to the next committee with a random $[0]_{2t}$ sharing.

While increasing the sharing degree to $2t$ facilitates an efficient transfer protocol, adopting degree- $2t$ sharings into computation will also increase communication even in the classical setting: 1. Multiplication becomes more expensive. To evaluate a multiplication gate $[x \cdot y]_{2t}$, we have to prepare a random beaver triple $([a]_{2t}, [b]_{2t}, [c]_{2t})$ in advance and open $x + a$ and $y + b$ publicly, then calculate $[z = x \cdot y]_{2t} = (x + a)(y + b) - (x + a)[b]_{2t} - (y + b)[a]_{2t} + [c]_{2t}$. The mere preparation of a Beaver triple is more costly than the entire multiplication evaluation in [10]. 2. The verification phase becomes more complicated and expensive. To check a $2t$ -degree $[x]_{2t}$, [4] uses a SPDZ-style authenticate form as $\llbracket x \rrbracket = ([x]_{2t}, [\Delta \cdot x]_{2t}, [\Delta]_{2t})$. This special form incurs significantly more computational overhead compared to efficient verification protocols with degree- t sharings.

Regrettably, the above overhead increases dramatically in the *maximal fluidity* setting. For instance, evaluating a multiplication gate now crosses at least 4 committees (take $C_{i-1}, C_i, C_{i+1}, C_{i+2}$ as an example), the processes can be summarized as: C_{i-1} generates random double sharings $([a]_t, [a]_{2t}), ([b]_t, [b]_{2t})$, locally computes $[c]_{2t} = [a]_t \cdot [b]_t$, and sends $([a]_{2t}, [b]_{2t}, [c]_{2t})$ to C_i ; C_i locally computes $[x + a]_{2t}$ and $[y + b]_{2t}$, sends all shares of $[x + a]_{2t}$ and $[y + b]_{2t}$ to $P_{king} \in C_{i+1}$, and transfers $([a]_{2t}, [b]_{2t}, [c]_{2t})$ to C_{i+1} ; Then, one designated party $P_{king} \in C_{i+1}$ reconstructs the secrets $x + a, y + b$ and sends them to C_{i+2} . C_{i+1} also reshare $([a]_{2t}, [b]_{2t}, [c]_{2t})$ to C_{i+2} ; Finally, C_{i+2} locally computes $[z = x \cdot y]_{2t} = (x + a)(y + b) - (x + a)[b]_{2t} - (y + b)[a]_{2t} + [c]_{2t}$. Throughout this process, messages like $[a]_{2t}, [b]_{2t}$ were reshared multiple times, causing massive inefficiency in communication. Similar to multiplication, the verification protocol of [4] requires the authenticated form $\llbracket x \rrbracket = ([x]_{2t}, [\Delta \cdot x]_{2t}, [\Delta]_{2t})$ to be preserved along with the computation, leading to a multiplicative increase in resharing, even neglecting the cost of verification itself.

Consequently, we attribute the inefficiencies of prior works to a compounding technical burden: primarily, the reliance on degree- $2t$ sharing multiplication protocols, which furthermore necessitates heavy communication redundancy for computation and verification—an issue that becomes particularly acute under the constraints of *maximal fluidity*. To address these systemic bottlenecks, our work explores the feasibility of adapting more efficient degree- t sharing protocols to the fluid model.

3.2 Towards Efficient Fluid MPC: Degree- t Resharing and Multiplication

In this section, we explore the feasibility of the DN protocol [10] in the *maximal fluidity* setting. Initially, an efficient protocol for transmitting degree- t shares is presented, which serves as the foundation for the fluid multiplication. Despite its efficiency, this construction is found to incur an inherent overhead when operating within the fluid model. To eliminate this, a separation-generation approach is introduced under *submaximal fluidity*. In particular, this approach allows us to align the communication cost with the DN protocol [10].

Efficiently Transferring Degree- t Sharings. Intuitively, transferring a degree- t sharing $[x]_t$ across committees might pose potential security risks, as an adversary could potentially gather several shares exceeding the privacy threshold. Furthermore, achieving such a transfer with linear communication complexity appears infeasible under *maximal fluidity*. Typically, linear overhead is realized by a designated party P_{king} , to reconstruct a masked value $x + r$ for subsequent distribution. However, this process requires both the initial and the succeeding committees to hold a random sharing $[r]_t$ of the same random mask r , which seemingly results in a circular dependency within the fluid model.

Contrary to these intuitions, inspired by [5], we observe that, by setting aside the components required for robustness, the underlying reshare protocol of [5] consists entirely of linear operations and remains independent of the sharing degree. Consequently, this construction can be naturally adapted into an efficient protocol for degree- t sharing transfer. The refined process can be summarized as follows:

1. Let $[x_1]_t^{C_i}, \dots, [x_{t+1}]_t^{C_i}$ be $(t+1)$ sharings held by C_i . Each party $P_j \in C_i$ distributes a degree- $2t$ packed secret sharing $\llbracket \mathbf{x}^j \rrbracket_{2t}$ to C_{i+1} , where $\mathbf{x}^j = (x_1^j, \dots, x_{t+1}^j)$ and x_p^j denotes the j -th share of $[x_p]_t^{C_i}$ held by P_j .
2. All parties in C_{i+1} locally computes $\llbracket \mathbf{x} \rrbracket_{2t}^{C_{i+1}} = \sum_{j=1}^n L_{j,2t} \cdot \llbracket \mathbf{x}^j \rrbracket_{2t}$, where $L_{j,2t}$ is the corresponding Lagrange coefficient.
3. Each party $P_j \in C_{i+1}$ distributes its share of $\llbracket \mathbf{x} \rrbracket_{2t}^{C_{i+1}}$ to C_{i+2} as a sharing $[x'^j]_t^{C_{i+2}}$.
4. For each $p \in [t+1]$, all parties in C_{i+2} can locally reconstruct $[x_p]_t^{C_{i+2}} = \sum_{j=1}^n L_{p,j,t} \cdot [x'^j]_t$, where $L_{p,j,t}$ is the corresponding Lagrange coefficient.

In particular, transferring $(t+1)$ sharings from C_i to C_{i+2} leads to a communication cost of $2n^2$ field elements. Note that computing $\llbracket \mathbf{x} \rrbracket_{2t}^{C_{i+1}}$ requires only

$t + 1$ packed secret sharings $\llbracket \mathbf{x}^j \rrbracket_{2t}$ since each x_p is fully determined by $(t + 1)$ shares of $[x_p]_t^{C_i}$. A modest optimization is to let only the first $(t + 1)$ parties in C_i distribute their shares. Hence, we obtain a transfer protocol that costs $\frac{n^2 + (t+1)n}{(t+1)n} \approx 3$ field elements per party per gate.

A Baseline Construction of Fluid Multiplication. After realizing efficient transfer, an efficient fluid multiplication protocol can be derived naturally by simply allocating each step of DN-Mult [10] to adjacent committees. The process can be described as:

1. The committee C_i holds the input sharings $[x]_t^{C_i}$ and $[y]_t^{C_i}$.
2. C_{i-1} can assist C_i in obtaining random double sharings $([r]_t^{C_i}, [r]_{2t}^{C_i})$.
3. C_i locally computes $[e]_{2t} = [x]_t^{C_i} \cdot [y]_t^{C_i} + [r]_{2t}^{C_i}$, sends all shares of $[e]_{2t}$ to one $P_{\text{king}} \in C_{i+1}$, and transfers $[r]_t$ to C_{i+2} .
4. $P_{\text{king}} \in C_{i+1}$ reconstructs the value e and then sends it to all parties in C_{i+2} .
5. All parties in C_{i+2} locally set $[z]_t^{C_{i+2}} := e - [r]_t^{C_{i+2}}$.

Compared to the original multiplication protocol [10], our only additional overhead during multiplication lies in the invocation of a transfer protocol once. Therefore, to evaluate a multiplication gate, each party needs to consume 9 field elements.

However, the additional transfer for $[r]_t$ is considered inherently unavoidable within the regime of *maximal fluidity*, as the variation of committee members precludes the possibility of two different committees holding shares of the same random mask without explicit communication. This fundamental bottleneck persists regardless of the optimization level achieved by the underlying transmission protocol, appearing as a structural limitation of the model itself.

Generating Split Random Double Sharings. To overcome the aforementioned structural bottleneck, we consider the *submaximal fluidity* setting, where a single intra-committee communication round is allowed. A re-examination of the DN protocol [10] reveals that the random double sharing $([r]_t, [r]_{2t})$ does not necessitate simultaneous generation. Specifically, the initial phase of the computation can proceed without the use of $[r]_t$. This decoupling implies that the two adjacent committees can be distributed seed sharings of distinct degrees, which are subsequently expanded locally into pairs of random double sharings.

To be more precise, each party $P_j \in C_i^1$ first samples a random seed s_j but only distributes $[s_j]_{2t}$ to C_i^2 . Subsequently, P_j continues to independently distribute $[s_j]_t$ to C_{i+1}^1 in the second communication round (i.e., C_i^2). After local expansion, C_i^1 and C_{i+1}^1 hold $[r]_{2t}$ and $[r]_t$ separately.

Achieving Original DN Efficiency in Fluid Settings. With our separation-generation approach, we construct a fluid protocol that attains the same efficiency as the classic DN protocol, which can be described as:

1. C_i^1 follows the separation-generation process.
2. C_i^2 holds two input sharings $[x]_t^{C_i}$ and $[y]_t^{C_i}$.

3. C_i^2 locally computes $[e]_{2t} = [x]_t^{C_i} \cdot [y]_t^{C_i} + [r]_{2t}$ and sends all shares of $[e]_{2t}$ to one designated party $P_{\text{king}} \in C_{i+1}$.
4. P_{king} reconstructs the value e and then sends it to all parties in C_{i+1}^2 .
5. All parties in C_{i+1}^2 locally set $[z]_t^{C_{i+1}} := e - [r]_t^{C_{i+1}}$.

Notably, the protocol introduces no extra overhead, with evaluation of each multiplication gate of only 6 field elements. Furthermore, it maintains computational consistency, ensuring that every committee obtains its input sharings within the second communication round to proceed with subsequent computations. For the resharing part, we show that the above transfer protocol can be simply modified to the *submaximal fluidity* setting, as detailed in 4.1.

3.3 More Optimizations to Reduce Communication

An additional advantage of adopting the DN paradigm is the potential for further efficiency gains via ATLAS [21] techniques, which is the state-of-the-art work in the classical setting. Crucially, the derivation from the DN to the ATLAS protocol can be achieved non-interactively, which allows the efficiency improvements to be realized in the fluid setting without any interference. Accordingly, the following presents two main optimizations: One reduces the communication cost of generating random double sharings, and another halves the overall communication rounds.

Relying on t -wise Independence. The first advancement is achieved by reducing half of the number of required random double sharings $([r]_t, [r]_{2t})$. A critical trick resides in letting each party in one committee act as a P_{king} when evaluating each n multiplication gates in parallel. For each $P_{\text{king}} \in C_{i+1}$ (resp., C_{i+1}^1 for *submaximal fluidity*), we let it distribute $[e = xy + r]_t$ instead of sending e to C_{i+2} (resp., C_{i+1}^2).

Then, for each honest P_{king} , we observe that there is no need to prepare a truly random $[r]_t$ since the final output sharing $[z]_t = [e]_t - [r]_t$ can be protected by $[e]_t$. There exists $(t + 1)$ honest and t corrupted P_{king} in a round of n multiplication gates, so it suffices to prepare t random $[r]_t$ for each corrupted P_{king} . The solution is to generate n pairs of double sharings such that any t pairs of them are independent and uniformly random, which is identified as the t -wise Independence property.

Fortunately, preparing n pairs of double sharings with t -wise Independence can be locally expanded by t random double sharings without interaction. The expansion can be computed as follows:

$$\begin{aligned} ([\hat{r}_1]_t, \dots, [\hat{r}_n]_t)^T &= M \cdot ([r_1]_t, \dots, [r_t]_t)^T \\ ([\hat{r}_1]_{2t}, \dots, [\hat{r}_n]_{2t})^T &= M \cdot ([r_1]_{2t}, \dots, [r_t]_{2t})^T \end{aligned}$$

, where M is a $n \times t$ a fixed hyper-invertible matrix.

Relying on t -wise Independence, we only need to prepare t instead of n pairs of random double sharings for n multiplication gates. Back to [10], generating

a pair of random double sharing costs 4 field elements. Hence, the amortized cost per double sharing is reduced to $4 \cdot \frac{t}{n} \approx 2$ field elements. During a round of n multiplications, a clever trick in the maximally fluid setting here is that C_i only needs to transfer t truly random sharings to C_{i+2} , which can locally expand them to n required sharings.

Then we can obtain an improved version of the multiplicative protocol by simply replacing the original construction with the above method to generate double sharings, leading to a communication complexity of 5.5 (resp., 4) field elements per party. Details can refer to Section 4.2.

Round Reduction. To reduce the communication rounds by half, we consider two layers of multiplication gates as a unit. Our goal is to allow C_{i+2} (resp., C_{i+1}^2 for *submaximal fluidity*) to locally evaluate Multiplications via beaver-like [1] operations.

The key technique is to leverage the so-called "beaver-friendly" form $[w]_t = u - [v]_t$, where u is public and $[v]_t$ is also held by the former committee. Our proposed fluid multiplication protocol indeed satisfies this requirement: For each output sharing $[x]_t$ of addition or relay gates, we can rewrite as $[x]_t = 0 - (-[x]_t)$, where $[x]_t$ is also held by the former committee C_i (resp., C_i^2). For each output sharing of multiplication gates $[z]_t = e - [r]_t$, C_i indeed holds $[r]_t$, a challenge arises in *submaximal fluidity*: C_i^2 do not inherently possess $[r]_t$ without incurring additional overhead, which will be addressed and optimized below.

Now given two input sharings $[z_1]_t = e_1 - [r_1]_t$ and $[z_2]_t = e_2 - [r_2]_t$ of a multiplication gate in the second layer, the former committee C_i (resp., C_i^2) can pre-compute $[r_1 \cdot r_2]_t$ in parallel with calculating $[z_1]_t$ and $[z_2]_t$ (this process can run with t -wise Independence for reducing communication), which enables C_{i+2} (resp., C_{i+1}^2) to locally compute $[z_1 \cdot z_2]_t = e_1 \cdot e_2 - e_1 \cdot [r_2]_t - e_2 \cdot [r_1]_t + [r_1 \cdot r_2]_t$. Consequently, the evaluation of all gates in the second layer can be performed non-interactively.

Moreover, an additional optimization derives from an observation that a pair of $([r]_t, [r]_{2t})$ is equivalent to a pair of $([r]_t, [0]_{2t})$ if r and $[0]_{2t}$ are uniformly random (i.e., $[e]_t = [x]_t \cdot [y]_t + [r]_{2t}$ can be replaced with $[e]_t = [x]_t \cdot [y]_t + [r]_t + [0]_{2t}$). Then, we can construct a batch of random $[0]_{2t}$ with t -wise Independence, reducing their generation cost by half. For the *submaximal fluidity* setting, the separation-generation approach is modified to let each $P_j \in C_i^2$ independently distributes $[s_j]_t$ rather than $[s_j]_{2t}$ to C_{i+1}^1 , which now ensures C_i^2 and C_{i+1}^1 to hold $[s_j]_t$ independently, followed by a local expansion to random $[r]_t$.

Building upon these optimizations, the amortized communication cost of a multiplication gate per party is reduced to 6.75 field elements (resp., 5.5), and of transferring a degree- t sharing per party 1.5 field elements since no additional transmission of all relay and addition gates in the second layer is required. Details can refer to Section 4.2.

3.4 Achieving Malicious Security

This section presents a verification protocol that enhances our semi-honest protocols to be secure against malicious adversaries. According to [9], they have

already proved that linear-based and weakly private fluid MPC protocols are secure up to additive attacks against malicious adversaries, which means that any attack operated by a malicious adversary is equivalent to injecting an additive error on the value of each wire in the circuit. Fortunately, [19] proved that the original DN protocol is linear-based and weakly private, which can be naturally extended to our protocols. Hence, to achieve malicious security, it is sufficient for us to guarantee our protocol's correctness in the presence of malicious adversaries.

Note that the correctness of the whole execution requires the following points:

- Each P_{king} sends the same value to the next committee during evaluating multiplication gates.
- The underlying secret remains consistent before and after each transfer.
- Each multiplication gate is computed correctly.

In our verification protocol, the first point can be checked by random linear combinations. For the remaining two points, following [9,26,4], the first committee generates a global random sharing $[r]_t$. For the sharing $[z]_t$ in each output wire, we use another multiplication to compute $[r \cdot z]_t$ along with $[z]_t$. Then, each committee C_i (resp., C_i^2 for *submaximal fluidity*) first calculates two random linear combinations on such paired sharings as:

$$\begin{aligned} [u'_i]_t &= \sum_{j=1}^{p_i} r_i^{j-1} \cdot [z_j]_t \\ [v'_i]_t &= \sum_{j=1}^{p_i} r_i^{j-1} \cdot [r \cdot z_j]_t \end{aligned}$$

, where $([z_j]_t)_{j \in [p_i]}$ is all output sharings held by C_i (resp., C_i^2) and r_i is randomly sampled. Immediately right after, C_i (resp., C_i^2) will receive a pair of accumulated check from C_{i-2} (resp., C_{i-1}^2), denoted as $([u_{i-2}]_t, [v_{i-2}]_t)$ (resp., $([u_{i-1}]_t, [v_{i-1}]_t)$), and update the current check as:

$$[u_i]_t = [u'_i]_t + [u_{i-2}]_t, [v_i]_t = [v'_i]_t + [v_{i-2}]_t$$

Finally, C_i transfer $([u_i]_t, [v_i]_t)$ and $[r]_t$ to continue the incremental check. Until the last committee completes the iterative check, we denote the final check sharings as $([u]_t, [v]_t, [r]_t)$. The last committee first opens $[r]_t$, aborts if $[r]_t$ is not consistent. Then they continue to open $[v]_t - r[u]_t$ and check whether the secret of $[v]_t - r[u]_t$ is zero.

4 Semi-Honest Protocol

This section presents our efficient fluid semi-honest protocols. Section 4.1 first illustrates how to realize efficient secure transfer and presents two baseline fluid multiplicative protocols. In the following section, we propose four optimized, efficient fluid multiplication protocols $\Pi_{\text{Velora-1}}$, $\Pi_{\text{lon-1}}$, $\Pi_{\text{Velora-2}}$, and $\Pi_{\text{lon-2}}$. Later,

Functionality $\mathcal{F}_{\text{semi-trans}}$

This Functionality runs with two committees C_i and C_{i+2} . C_i holds $(t+1)$ sharings $[x_1]_t, \dots, [x_{t+1}]_t$. This Functionality operates as follows:

1. For $p \in [t+1]$, upon receiving the shares of $[x_p]_t$ from honest parties in C_i , $\mathcal{F}_{\text{semi-trans}}$ reconstructs the whole sharing $[x_p]_t$ secret x and sends corrupted shares of $[x_p]_t$ to the adversary.
2. For $p \in [t+1]$, $\mathcal{F}_{\text{semi-trans}}$ receives t shares $(\bar{x}_p^j)_{\{j \in \text{Corr}_{C_{i+2}}\}}$ from the adversary S .
3. For $p \in [t+1]$, based on t corrupted shares $(\bar{x}_p^j)_{\{j \in \text{Corr}_{C_{i+2}}\}}$ and the secret x_p , $\mathcal{F}_{\text{semi-trans}}$ computes the sharing $[x_p]_t$ and sends honest shares of it to $H_{C_{i+2}}$.

Fig. 2: Secure transfer in fluid setting

we introduce the main protocol that realizes $\mathcal{F}_{\text{dabb}}$ in the presence of a semi-honest adversary in the last section. For simplicity, we assume that all committees, including the client committee, are formed by exactly $n = 2t + 1$ parties and allow the adversary to control t parties in each committee.

4.1 Baseline Fluid Multiplication

Efficiently Transfer Degree- t Sharings. Here, we first implement the transfer protocol with linear communication overhead. We formalize the transmission Functionality $\mathcal{F}_{\text{semi-trans}}$ shown in Figure 2. At a high level, our goal is to transfer a batch of degree- t sharings held by C_i to C_{i+2} . Each party $P_j \in C_i$ distributes a packed secret sharing with setting its shares of degree- t sharings as the secret vector of the PSS to C_{i+1} . Then, C_{i+1} can locally compute a PSS containing all underlying secrets of degree- t sharings via interpolation, and distribute each share of this PSS to C_{i+2} . Subsequently, C_{i+2} recovers each sharing to be transferred through interpolation. The overall operation is described as Π_{trans} shown in Figure 3.

Communication Cost of Π_{trans} . Transferring $(t+1)$ degree- t sharings in Π_{trans} requires the communication cost of $n^2 + (t+1)n$ field elements. Therefore, the amortized cost of secure transfer per party is $\frac{n^2 + (t+1)n}{(t+1)n} \approx 3$ field elements.

Adapting to *submaximal fluidity*. Notably, Π_{trans} is also compatible with the *submaximal fluidity* setting and can be adapted as: C_i^2 holds $(t+1)$ input sharings $[x_1]_t, \dots, [x_{t+1}]_t$. Then, each party $P_j \in C_i^2$ distributes $\llbracket \mathbf{x}^j \rrbracket_{2t}$ to C_{i+1}^1 . All parties in C_{i+1}^1 reconstruct the sharing $\llbracket \mathbf{x} \rrbracket_{2t}$ and distributes its share to C_{i+1}^2 . Finally, all parties in C_{i+1}^2 recover $(t+1)$ initial sharings locally.

Theorem 1. *Protocol Π_{trans} shown in Figure 3 securely realizes Functionality $\mathcal{F}_{\text{semi-trans}}$ shown in Figure 2 with information-theoretic security in the presence of a semi-honest adversary controlling t corrupted parties in each committee.*

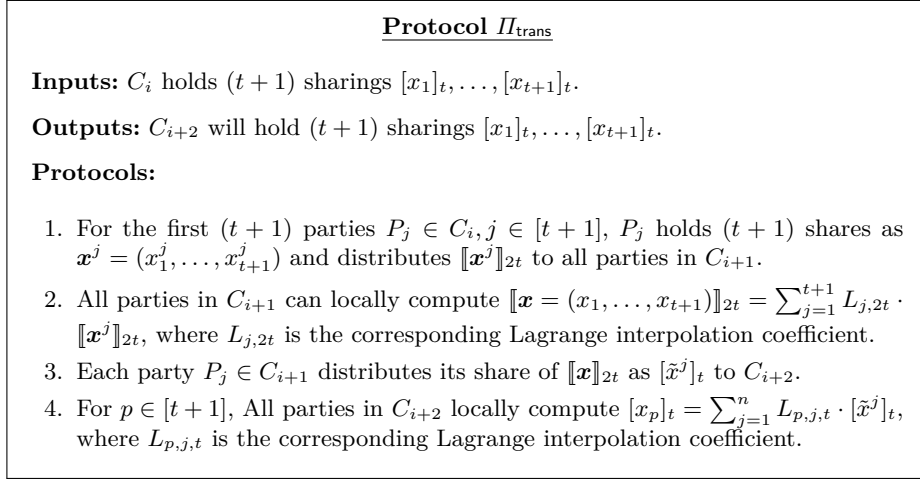


Fig. 3: Protocol for secure transfer

Proof. We leave the formal proof to Appendix E.

Adapting to *submaximal fluidity*. Theorem 1 remains applicable to the *submaximal fluidity* setting, where the interaction is restricted to only two committees C_i and C_{i+1} .

Realizing Random Double Sharings in Fluid Model. We proceed to construct a protocol to generate random double sharings in the *maximal fluidity* setting. This protocol can be simply modified from its original version in [10]. Concretely, the Functionality $\mathcal{F}_{\text{doublerand}}$ shown in Figure 4 generates a pair of random double sharing $([r]_t, [r]_{2t})$ to a committee C_i , which can be realized by our protocol $\Pi_{\text{doublerand}}$ presented in Figure 5.

Communication Cost of $\Pi_{\text{doublerand}}$. Generating $(t + 1)$ such random double sharings consumes $2n^2$ field elements. Therefore, preparing one pair of random double sharing in the fluid model requires $\frac{2n^2}{t+1} \approx 4$ field elements of communication.

Theorem 2. *Protocol $\Pi_{\text{doublerand}}$ shown in Figure 5 securely realizes Functionality $\mathcal{F}_{\text{doublerand}}$ shown in Figure 4 with information-theoretic security in the presence of a semi-honest adversary controlling t corrupted parties in each committee.*

Proof. A detailed proof is presented in Appendix E.

After realizing $\mathcal{F}_{\text{doublerand}}$ and $\mathcal{F}_{\text{semi-trans}}$, we can naturally obtain a fluid-DN protocol $\Pi_{\text{dn-mult}}$ (shown in Figure 6), with additional invoking $\mathcal{F}_{\text{semi-trans}}$ once to transfer a random $[r]_t$ from C_i to C_{i+2} . Indeed, the amortized communication cost per multiplication gate per party increases to 9 field elements.

Functionality $\mathcal{F}_{\text{doublerand}}$

This Functionality runs with the committee C_i and operates as follows:

1. Upon receiving the shares of $(r_u)_{\{u \in \text{Corr}_{C_i}\}}$ and $(r_v)_{\{v \in \text{Corr}_{C_i}\}}$ from the adversary. $\mathcal{F}_{\text{doublerand}}$ views the first and the second as the corrupted shares of degree- $2t$ sharing and degree- t sharing in C_i separately. $\mathcal{F}_{\text{doublerand}}$ executes the following:
 - $\mathcal{F}_{\text{doublerand}}$ randomly samples a r and reconstructs a degree- t sharing $[r]_t$ based on r and t shares as $(r_v)_{\{v \in \text{Corr}_{C_i}\}}$.
 - $\mathcal{F}_{\text{doublerand}}$ randomly samples another t field elements as the shares of the degree- $2t$ sharing held by honest parties, and reconstructs the whole $[r]_{2t}$ based on r , t shares of honest parties and t shares of corrupted parties.
2. Finally, $\mathcal{F}_{\text{doublerand}}$ sends the shares of $([r]_t, [r]_{2t})$ to honest parties in C_i .

Fig. 4: Double sharings in fluid setting

Protocol $\Pi_{\text{doublerand}}$

Inputs: Given a $(t + 1) \times n$ Vandermonde matrix M .

Outputs: C_i obtains $(t + 1)$ random pair of double sharings $([r_u]_t, [r_u]_{2t})_{u \in [t+1]}$.

Protocols:

1. For each party $P_j \in C_{i-1}$, P_j randomly samples an s_j and distributes $([s_j]_t, [s_j]_{2t})$ to C_i .
2. All parties in C_i locally compute as follows:

$$([r_1]_d, \dots, [r_{t+1}]_d)^T = M \cdot ([s_1]_d, \dots, [s_n]_d)^T$$

for $d \in \{t, 2t\}$.

3. All parties in C_i outputs $(t + 1)$ random pair of double sharings $([r_u]_t, [r_u]_{2t})_{u \in [t+1]}$.

Fig. 5: Protocol for generating double sharings

Theorem 3. *Protocol $\Pi_{\text{dn-mult}}$ shown in Figure 6 securely realizes Functionality $\mathcal{F}_{\text{semi-mult}}$ shown in Figure 30 in the $(\mathcal{F}_{\text{doublerand}}, \mathcal{F}_{\text{semi-trans}})$ -hybrid with information-theoretic security in the presence of a semi-honest adversary controlling t corrupted parties in each committee.*

Proof. The formal proof is shown in Appendix E.

Given that the extra transmission of $[r]_t$ is inevitable under *maximal fluidity*, the *submaximal fluidity* setting is adopted to circumvent this limitation and achieve efficiency parity with the original DN protocol.

Protocol $\Pi_{\text{dn-mult}}$

Inputs: C_i holds two input sharings $[x]_t$ and $[y]_t$.

Outputs: C_{i+2} obtains $[z = x \cdot y]_t$.

Protocols:

1. C_i invokes $\mathcal{F}_{\text{doublerand}}$ to obtain a pair of random $([r]_t, [r]_{2t})$.
2. All parties in C_i locally compute $[x \cdot y + r]_{2t}$ and send the shares of it to a $P_{\text{king}} \in C_{i+1}$.
3. C_i invokes $\mathcal{F}_{\text{semi-trans}}$ to transfer $[r]_t$ to C_{i+2} .
4. The $P_{\text{king}} \in C_{i+1}$ reconstructs the secret $e = x \cdot y + r$ and sends it to all parties in C_{i+2} .
5. All parties in C_{i+2} locally compute $[z]_t = e - [r]_t$.

Fig. 6: **Protocol for fluid DN-Multiplication**

Protocol $\Pi_{\text{split-doublerand}}$

Inputs: Let M be a fixed $(t+1) \times n$ Vandermonde matrix.

Outputs: Generating $(t+1)$ double sharings $([r_j]_t, [r_j]_{2t})_{j \in [t+1]}$ with C_i^2 holding $[r_j]_{2t}$ and C_{i+1}^1 holding $[r_j]_t$.

Protocols:

1. For each $P_j \in C_i^1$, P_j randomly samples $s_j \in \mathbb{F}$ and distributes $[s_j]_{2t}$ to C_i^2 .
2. All parties in C_i^2 locally compute

$$([r_1]_{2t}, \dots, [r_{t+1}]_{2t})^T = M \cdot ([s_1]_{2t}, \dots, [s_n]_{2t})^T$$

and distribute $([s_j]_t)_{j \in [n]}$ to C_{i+1}^1 .

3. All parties in C_{i+1}^1 locally compute

$$([r_1]_t, \dots, [r_{t+1}]_t)^T = M \cdot ([s_1]_t, \dots, [s_n]_t)^T$$

Fig. 7: **Protocol for generating split double sharings**

Generating Split Random Double Sharings. In the *submaximal fluidity* setting, our solution is to generate split random sharings for two adjacent committees C_i and C_{i+1} respectively, which eliminates the need to repeatedly transfer sharings. The implementation of this idea is presented in Figure 7, with the corresponding Functionality shown in Figure 8.

Theorem 4. *Protocol $\Pi_{\text{split-doublerand}}$ shown in Figure 7 securely realizes Functionality $\mathcal{F}_{\text{split-doublerand}}$ shown in Figure 8 with information-theoretic security in*

Functionality $\mathcal{F}_{\text{split-doublerand}}$

This Functionality runs with two adjacent committees C_i and C_{i+1} . This Functionality operates as follows:

1. Upon receiving the shares of $r_{\ell\{P_\ell \in \text{Corr}_{C_i}\}}$ and $r_{j\{P_j \in \text{Corr}_{C_{i+1}}\}}$ from corrupted parties. $\mathcal{F}_{\text{split-doublerand}}$ views the first as the shares for degree- $2t$ sharing in C_i and the second as the shares for degree- t sharing in C_{i+1} . $\mathcal{F}_{\text{split-doublerand}}$ executes the following:
 - $\mathcal{F}_{\text{split-doublerand}}$ randomly samples a r and reconstructs the whole $[r]_t$ based on r together with t corrupted parties $r_{j\{P_j \in \text{Corr}_{C_{i+1}}\}}$.
 - $\mathcal{F}_{\text{split-doublerand}}$ randomly samples t elements as the shares of honest parties in degree- $2t$ sharing and reconstructs the whole $[r]_{2t}$ based on r , t shares of honest parties and t shares of corrupted parties $r_{\ell\{P_\ell \in \text{Corr}_{C_i}\}}$.
2. Finally, $\mathcal{F}_{\text{split-doublerand}}$ sends the shares of $([r]_t, [r]_{2t})$ to honest parties among two committees.

Fig. 8: Split random double sharings in Fluid Setting

Protocol $\Pi_{\text{split-mult}}$

Protocols:

1. C_i and C_{i+1} call $\mathcal{F}_{\text{split-doublerand}}$ to obtain $([r]_t, [r]_{2t})$.
2. All parties in C_i^2 can locally compute $[x \cdot y + r]_{2t}$ and send the shares to P_{king} in C_{i+1}^1 .
3. P_{king} in C_{i+1}^1 reconstructs the secret $x \cdot y + r$ and sends to other parties in C_{i+1}^2 .
4. All parties in C_{i+1}^2 locally compute $[z]_t = x \cdot y + r - [r]_t$.

Fig. 9: Protocol for split-Multiplication

the presence of a semi-honest adversary controlling t corrupted parties in each committee.

Proof. The formal proof refers to Appendix E.

Attaining DN-Level Efficiency under *Submaximal Fluidity*. With constructed $\Pi_{\text{split-doublerand}}$, a new multiplication protocol $\Pi_{\text{split-mult}}$ is illustrated in Figure 9, achieving the same efficiency as the DN protocol (i.e., costing 6 field elements per multiplication gate). Similar to Theorem 3, we present a version of $\Pi_{\text{split-mult}}$ in Appendix E.

Protocol $\Pi_{\text{expand-sharings}}$

Inputs: A committee C_i holds t pairs of random double sharings $([r_u]_t, [r_u]_{2t})_{u \in [t]}$. Let M_1 be an $n \times t$ hyper-invertible matrix.

Outputs: C_i obtains n pairs of sharings $([\hat{r}_v]_t, [\hat{r}_v]_{2t})_{v \in [n]}$ with t -wise Independence.

Protocols:

1. All parties in C_i locally compute

$$\begin{aligned} ([\hat{r}_1]_t, \dots, [\hat{r}_n]_t)^T &= M_1 \cdot ([r_1]_t, \dots, [r_t]_t)^T, \\ ([\hat{r}_1]_{2t}, \dots, [\hat{r}_n]_{2t})^T &= M_1 \cdot ([r_1]_{2t}, \dots, [r_t]_{2t})^T \end{aligned}$$

2. C_i outputs n pairs of double sharings $([\hat{r}_v]_t, [\hat{r}_v]_{2t})_{v \in [n]}$.

Fig. 10: **Protocol for expanding random sharings**

4.2 Efficient Fluid Multiplication

In this section, we extend the ATLAS techniques from [21] to our fluid protocols $\Pi_{\text{dn-mult}}$ and $\Pi_{\text{split-mult}}$ for further improving efficiency. The following two primary techniques are introduced to enhance the performance of the baseline protocols:

Relying on Rotating P_{king} and t -wise Independence. The first advancement is achieved by rotating each party as a P_{king} in a batch of n multiplication gates. We initially specify that each P_{king} distributes $[e = xy + r]_t$ instead of directly sending e to the next committee.

Indeed, privacy of the output sharing $[z]_t = [e]_t - [r]_t$ held by the next committee is guaranteed by $[e]_t$ distributed from an honest P_{king} , meaning that a random $[r]_t$ is unnecessary for this P_{king} . Hence, in a round of n multiplication gates, we only need to prepare t truly random pairs of double sharings to the unknown t corrupted parties. Fortunately, such n double sharings with this property can be locally expanded by t random double sharings, which is illustrated in Figure 10.

Lemma 1. *The output double sharings in $\Pi_{\text{expand-sharings}}$ shown in Figure 10 exhibit the t -wise Independence property.*

Proof. A formal proof refers to Appendix F.

Subsequently, we can perform parallel efficient evaluation of n multiplication gates, and first formalize this operation as the Functionality $\mathcal{F}_{\text{batch-mult}}$ shown in Figure 11 under *maximal fluidity*. Indeed, we construct a protocol $\Pi_{\text{Velora-1}}$ integrating our conception to realize $\mathcal{F}_{\text{batch-mult}}$. A key technique in $\Pi_{\text{Velora-1}}$ is that all parties in C_i invoke $\mathcal{F}_{\text{semi-trans}}$ on each $[r]_t$ instead of $[\hat{r}]_t$, which reduces the number of transferring sharings to C_{i+2} . $\Pi_{\text{Velora-1}}$ is presented in Figure

Functionality $\mathcal{F}_{\text{batch-mult}}$

This Functionality runs with two adjacent committees C_i and C_{i+2} . C_i holds a batch of N input sharings $([x_p]_t, [y_p]_t)_{p \in [N]}$.

1. $\mathcal{F}_{\text{batch-mult}}$ receives N from all parties in C_i .
2. For $p = 1, \dots, N$, $\mathcal{F}_{\text{batch-mult}}$ calls $\mathcal{F}_{\text{semi-mult}}$ on $[x_p]_t$ and $[y_p]_t$.

Fig. 11: **Batch-Mult in fluid setting**

Protocol $\Pi_{\text{Velora-1}}$

Inputs: Given $N \geq n$, C_i hold a batch of N input sharings $([x_p]_t, [y_p]_t)_{p \in [N]}$.

Outputs: C_{i+2} obtains N sharings $([z_p]_t = [x_p \cdot y_p]_t)_{p \in [N]}$.

Protocols:

1. C_i calls $\mathcal{F}_{\text{doublerand}}$ to obtain $\frac{N \cdot t}{n}$ pairs of random double sharings, and invokes $\Pi_{\text{expand-sharings}}$ on these sharings to obtain $\frac{N}{n}$ groups of n double sharings with t -wise Independence, denoted as $\{([\hat{r}_{(u-1)n+1}]_t, [\hat{r}_{(u-1)n+1}]_{2t}), \dots, ([\hat{r}_{un}]_t, [\hat{r}_{un}]_{2t})\}_{u \in [\frac{N}{n}]}$.
2. For each $u \in [\frac{N}{n}]$ and $j \in [n]$, all parties in C_i compute $[e_{(u-1)n+j}]_{2t} = [x_{(u-1)n+j}]_t \cdot [y_{(u-1)n+j}]_t + [\hat{r}_{(u-1)n+j}]_{2t}$, and send shares of $[e_{(u-1)n+j}]_{2t}$ to $P_j \in C_{i+1}$.
3. All parties in C_i invoke $\mathcal{F}_{\text{semi-trans}}$ on $\frac{N \cdot t}{n}$ degree- t random sharings.
4. For each $u \in [\frac{N}{n}]$ and $j \in [n]$, each $P_j \in C_{i+1}$ reconstructs $e_{(u-1)n+j}$, and distributes $[e_{(u-1)n+j}]_t$ to C_{i+2} .
5. All parties in C_{i+2} first invoke $\Pi_{\text{expand-sharings}}$ on received sharings from $\mathcal{F}_{\text{semi-trans}}$ to obtain $\{[\hat{r}_{(u-1)n+1}]_t, \dots, [\hat{r}_{un}]_t\}_{u \in [\frac{N}{n}]}$. For each $u \in [\frac{N}{n}]$ and $j \in [n]$, C_{i+2} locally computes $[z_{(u-1)n+j}]_t = [e_{(u-1)n+j}]_t - [\hat{r}_{(u-1)n+j}]_t$.

Fig. 12: **Velora-1 Multiplication**

12. For *submaximal fluidity* setting, a protocol $\Pi_{\text{lon-1}}$ shown in Figure 13 is constructed to realize $\mathcal{F}_{\text{batch-mult}}$ (its submaximal version).

Communication Cost of $\Pi_{\text{Velora-1}}$. Evaluating N multiplication gates in $\Pi_{\text{Velora-1}}$ costs $4Nt + 3Nt + 2Nn = 7Nt + 2Nn$ field elements. Therefore, the amortized communication cost per multiplication gate per party in $\Pi_{\text{Velora-1}}$ is $\frac{7Nt + 2Nn}{N \cdot n} \approx 5.5$ field elements.

Communication Cost of $\Pi_{\text{lon-1}}$. Given $N \geq n$ multiplication gates, the total communication totals $\frac{N}{n} \cdot [2 \times n^2 + n \times (n + n)] = 4Nn$ elements. Therefore, each multiplication gate costs $\frac{4Nn}{N \cdot n} = 4$ elements per party.

Protocol $\Pi_{\text{lon-1}}$

Inputs: Given $N \geq n$, C_i^2 holds N input sharings $([x_p]_t, [y_p]_t)_{p \in [N]}$.

Outputs: C_{i+1}^2 obtains a set of sharings $([z_p = x_p \cdot y_p]_t)_{p \in [N]}$.

Protocols:

1. C_i and C_{i+1} call $\mathcal{F}_{\text{split-doubling}}$ to obtain $\frac{N \cdot t}{n}$ pairs of random double random sharings.
2. Firstly, C_i^2 invokes $\Pi_{\text{expand-sharings}}$ to obtain t -wise Independence sharings $([\tilde{r}_{1,\ell}]_{2t}, \dots, [\tilde{r}_{n,\ell}]_{2t})_{\ell \in [\frac{N}{n}]}$. For each round $\ell = 1, \dots, \frac{N}{n}$, C_{i+1}^2 locally computes a set of n sharings $([x_{1,\ell} \cdot y_{1,\ell} + \tilde{r}_{1,\ell}]_{2t}, \dots, [x_{n,\ell} \cdot y_{n,\ell} + \tilde{r}_{n,\ell}]_{2t})$, and sends $[x_{j,\ell} \cdot y_{j,\ell} + \tilde{r}_{j,\ell}]_{2t}$ to $P_j \in C_{i+1}^1$.
3. All parties invoke $\Pi_{\text{expand-sharings}}$ to obtain t -wise Independence sharings $([\tilde{r}_{1,\ell}]_t, \dots, [\tilde{r}_{n,\ell}]_t)_{\ell \in [\frac{N}{n}]}$. For each round $\ell = 1, \dots, \frac{N}{n}$, all parties in C_{i+1}^1 behaves like a P_{king} sequentially. $P_j \in C_{i+1}^1$ reconstructs the secret $e_{j,\ell} = x_{j,\ell} \cdot y_{j,\ell} + \tilde{r}_{j,\ell}$ and shares $[e_{j,\ell}]_t$ to C_{i+1}^2 .
4. All parties in C_{i+1}^2 locally compute $[z_i]_t = [e_{i,\ell}]_t - [\tilde{r}_{i,\ell}]_t$ for $i \in [n], \ell \in [\frac{N}{n}]$.

Fig. 13: lon-1 Multiplication

Theorem 5. *Protocol $\Pi_{\text{Velora-1}}$ shown in Figure 12 securely realizes Functionality $\mathcal{F}_{\text{batch-mult}}$ shown in Figure 11 in the $(\mathcal{F}_{\text{doubling}}, \mathcal{F}_{\text{semi-trans}})$ -hybrid model with information-theoretic security in the presence of a semi-honest adversary controlling t corrupted parties in each committee.*

Proof. A formal proof refers to Appendix F.

Theorem 6. *Protocol $\Pi_{\text{lon-1}}$ shown in Figure 13 securely realizes Functionality $\mathcal{F}_{\text{batch-mult}}$ shown in Figure 11 (its submaximal version) in the $\mathcal{F}_{\text{split-doubling}}$ -hybrid model with information-theoretic security in the presence of a semi-honest adversary controlling t corrupted parties in each committee.*

Proof. A formal proof refers to Appendix F.

Round Reduction. In this part, we consider a two-layer structure, where we wish to enable non-interactive evaluation of the second-layer multiplication gates. For the first layer, C_i uses $\Pi_{\text{dn-mult}}$ to evaluate the first-layer multiplication gates and invokes Π_{trans} to transfer output sharings of relay gates and addition gates. Then, for each input sharing $[z]_t$ to the next layer held by C_{i+2} , $[z]_t$ is of the form $[z]_t = e - [r]_t$, where e is public and $[r]_t$ is also held by C_i . Consequently, given two input sharings $[z_1]_t = e_1 - [r_1]_t$ and $[z_2]_t = e_2 - [r_2]_t$ of a multiplication gate held by C_{i+2} , C_i can invoke $\Pi_{\text{Velora-1}}$ to compute $[r_1 \cdot r_2]_t$ in parallel with $[z_1]_t$ and $[z_2]_t$, which facilitates C_{i+2} to calculate $[z_1 \cdot z_2]_t$ locally.

Another optimization is realized by a new observation that a pair of random $([r]_t, [r]_{2t})$ is equivalent to a pair of $([r]_t, [0]_{2t})$ with a random $[0]_{2t}$. With the

Protocol $\Pi_{\text{expand-zero}}$

Inputs: A committee C_i holds t random zero sharings $([0_{(j)}]_{2t})_{j \in [t]}$. Let M_1 be an $n \times t$ hyper-invertible matrix.

Outputs: C_i obtains n zero sharings $([\hat{0}_u]_{2t})_{u \in [n]}$ satisfying t -wise Independence.

Protocols:

1. All parties in C_i locally compute

$$([\hat{0}_1]_{2t}, \dots, [\hat{0}_n]_{2t})^T = M_1 \cdot ([0_{(1)}]_{2t}, \dots, [0_{(t)}]_{2t})^T.$$

2. C_i outputs n zero sharings $([\hat{0}_u]_{2t})_{u \in [n]}$.

Fig. 14: **Protocol for expanding zero sharings**

similar argument of $\Pi_{\text{Velora-1}}$, in a round of n multiplication gates, C_i turns to compute $[e]_{2t} = [x]_t \cdot [y]_t + [r]_t + [0]_{2t}$. Therefore, we only need to prepare n zero sharings with t -wise Independence.

In summary, we construct a protocol $\Pi_{\text{expand-zero}}$ shown in Figure 14 to locally expand zero sharings. Then, we describe an improved multiplicative protocol $\Pi_{\text{batch-mult}}$ in the fluid setting, which is shown in Figure 15. Later, we integrate the techniques proposed in this section to present the final efficient evaluation protocol, which is formalized as $\Pi_{\text{Velora-2}}$ and illustrated in Figure 16.

Lemma 2. *The output zero sharings in $\Pi_{\text{expand-zero}}$ shown in Figure 14 exhibit the t -wise Independence property.*

Proof. A formal proof refers to Appendix F.

Communication Cost of $\Pi_{\text{batch-mult}}$. Evaluating N multiplication gates in $\Pi_{\text{batch-mult}}$ costs $2Nt + 2Nn + 3Nn + 2Nn = 2nt + 7Nn$ field elements. Therefore, the amortized communication cost per multiplication gate per party in $\Pi_{\text{Velora-1}}$ is $\frac{2Nt + 7Nn}{N \cdot n} \approx 8$ field elements.

Corollary 1. *Protocol $\Pi_{\text{batch-mult}}$ shown in Figure 15 securely realizes Functionality $\mathcal{F}_{\text{batch-mult}}$ shown in Figure 11 in the $(\mathcal{F}_{\text{rand}}, \mathcal{F}_{\text{zero}}, \mathcal{F}_{\text{semi-trans}})$ -hybrid model with information-theoretic security in the presence of a semi-honest adversary controlling t corrupted parties in each committee.*

Proof. This Corollary can be proved in the same way as that for Theorem 5. $\square$

Corollary 2. *Protocol $\Pi_{\text{Velora-2}}$ shown in Figure 16 securely realizes Functionality $\mathcal{F}_{\text{batch-mult}}$ shown in Figure 11 in the $(\mathcal{F}_{\text{rand}}, \mathcal{F}_{\text{doublerand}}, \mathcal{F}_{\text{zero}}, \mathcal{F}_{\text{semi-trans}})$ -hybrid model with information-theoretic security in the presence of a semi-honest adversary controlling t corrupted parties in each committee.*

Protocol $\Pi_{\text{batch-mult}}$

Inputs: Given $N \geq n$, C_i hold a batch of N input sharings $([x_p]_t, [y_p]_t)_{p \in [N]}$.

Outputs: C_{i+2} obtains N sharings $([z_p = x_p \cdot y_p]_t)_{p \in [N]}$.

Protocols:

1. C_i invokes $\mathcal{F}_{\text{rand}}$ to obtain N random degree- t sharings. C_i invokes $\mathcal{F}_{\text{zero}}$ to obtain $\frac{N \cdot t}{n}$ random zero degree- $2t$ sharings, and invokes $\Pi_{\text{expand-sharings}}$ on these sharings to obtain $\frac{N}{n}$ groups of n zero degree- $2t$ sharings with t -wise Independence, denoted as $\{[\hat{0}_{(u-1)n+1}]_{2t}, \dots, [\hat{0}_{un}]_{2t}\}_{u \in [\frac{N}{n}]}$.
2. For each $u \in [\frac{N}{n}]$ and $j \in [n]$, all parties in C_i compute $[e_{(u-1)n+j}]_{2t} = [x_{(u-1)n+j}]_t \cdot [y_{(u-1)n+j}]_t + [r_{(u-1)n+j}]_t + [\hat{0}_{(u-1)n+j}]_{2t}$, and send shares of $[e_{(u-1)n+j}]_{2t}$ to $P_j \in C_{i+1}$.
3. All parties in C_i invoke $\mathcal{F}_{\text{semi-trans}}$ on N degree- t random sharings.
4. For each $u \in [\frac{N}{n}]$ and $j \in [n]$, each $P_j \in C_{i+1}$ reconstructs $e_{(u-1)n+j}$, and sends $e_{(u-1)n+j}$ to all parties in C_{i+2} .
5. For each $u \in [\frac{N}{n}]$ and $j \in [n]$, C_{i+2} locally computes $[z_{(u-1)n+j}]_t = e_{(u-1)n+j} - [r_{(u-1)n+j}]_t$.

Fig. 15: Protocol for batch-mult

Protocol $\Pi_{\text{Velora-2}}$

Inputs: Three adjacent committees C_i , C_{i+1} and C_{i+2} want to evaluate all gates in the $(2i-1)$ -th layer and the $(2i)$ -th layer. Let N_{2i-1} and N_{2i} be the number of multiplication gates in the $(2i-1)$ -th layer and $(2i)$ -th layer (Assuming that $N_{2i-1}, N_{2i} \geq n$).

Protocols:

1. C_i invokes $\Pi_{\text{batch-mult}}$ to evaluate all multiplication gates in the $(2i-1)$ -th layer.
2. C_i invokes $\Pi_{\text{Velora-1}}$ to compute $[a \cdot b]_t$ for all paired inputs $(u - [a]_t, v - [b]_t)$ to the multiplication gates in the $(2i)$ -th layer. Note that C_i precisely holds all paired sharings $([a]_t, [b]_t)$, where $[a]_t$ may be a random sharing or an input sharing held by C_i .
3. C_{i+1} follows $\Pi_{\text{batch-mult}}$ and $\Pi_{\text{Velora-1}}$.
4. After receiving all output sharings, C_{i+2} locally evaluate addition gates and relay gates. C_{i+2} also locally evaluate multiplication gates as $[z_1 \cdot z_2]_t = (u - [a]_t) \cdot (v - [b]_t) = u \cdot v + ([a \cdot b]_t - u \cdot [b]_t - v \cdot [a]_t)$.

Fig. 16: Velora-2 Multiplication

Functionality $\mathcal{F}_{\text{split-samerand}}$

This Functionality runs with two adjacent committees C_i and C_{i+1} . This Functionality operates as follows:

1. Upon receiving the shares of $r_u\{P_u \in \text{Corr}_{C_i}\}$ and $r_j\{P_j \in \text{Corr}_{C_{i+1}}\}$ from corrupted parties. $\mathcal{F}_{\text{split-samerand}}$ views the first as the shares for t -degree sharing in C_i and the second as the shares for t -degree sharing in C_{i+1} . $\mathcal{F}_{\text{split-samerand}}$ execute the following:
 - $\mathcal{F}_{\text{split-samerand}}$ randomly samples a r and reconstructs the whole $[r]_t^{C_i}$ based on r together with $r_u\{P_u \in \text{Corr}_{C_i}\}$, similarly with $[r]_t^{C_{i+1}}$ based on r together with $r_u\{P_u \in \text{Corr}_{C_i}\}$.
2. Finally, $\mathcal{F}_{\text{split-samerand}}$ sends the shares of $([r]_t^{C_i})$ to honest parties in C_i and $[r]_t^{C_{i+1}}$ to honest parties in C_{i+1} .

Fig. 17: Same rand sharings in fluid setting

Proof. This proof follows entirely from Theorem 5 and Corollary 1. □

Communication Cost of $\Pi_{\text{Velora-2}}$. The protocol $\Pi_{\text{Velora-2}}$ can be considered as making one invocation of Protocol $\Pi_{\text{Velora-1}}$ and one of Protocol $\Pi_{\text{batch-mult}}$. If we assume that $N_{2i-1} \approx N_{2i}$, the amortized communication cost per multiplication gate per party in $\Pi_{\text{Velora-2}}$ is about $\frac{8+5.5}{2} = 6.75$ field elements.

For the *submaximal fluidity setting*, to minimize additional transmissions, a new generation protocol $\Pi_{\text{split-samerand}}$ (shown in Figure 18) is proposed that enables two adjacent committees C_i and C_{i+1} to obtain the same random sharing $[r]_t$, with its corresponding Functionality $\mathcal{F}_{\text{split-samerand}}$ shown in Figure 17. Finally, we construct a protocol $\Pi_{\text{lon-2}}$ (shown in Figure 19) to realize multiplication.

Theorem 7. *Protocol $\Pi_{\text{split-samerand}}$ shown in Figure 18 securely realizes Functionality $\mathcal{F}_{\text{split-samerand}}$ shown in Figure 17 with information-theoretic security in the presence of a semi-honest adversary controlling t corrupted parties in each committee.*

Proof. A formal proof refers to Appendix F.

Corollary 3. *Protocol $\Pi_{\text{lon-2}}$ shown in Figure 19 securely realizes Functionality $\mathcal{F}_{\text{batch-mult}}$ shown in Figure 11 in the $(\mathcal{F}_{\text{split-samerand}}, \mathcal{F}_{\text{split-doublerand}}, \mathcal{F}_{\text{zero}})$ -hybrid model with information-theoretic security in the presence of a semi-honest adversary controlling t corrupted parties in each committee.*

Proof. Similar to Corollary 2, this proof follows Theorem 6 and Theorem 7. □

Communication Cost of $\Pi_{\text{lon-2}}$. Similar analysis to $\Pi_{\text{Velora-2}}$, the amortized cost per party of evaluating a multiplication gate is $\frac{7+4}{2} = 5.5$ elements if $N_{2i-1} \approx N_{2i}$.

Protocol $\Pi_{\text{split-samerand}}$

Inputs: Let M be a fixed $(t+1) \times n$ Vandermonde matrix.

Outputs: Generating $(t+1)$ random degree- t sharings $[r_j]_t$ held by C_i^2 and C_{i+1}^1 independently.

Protocols:

1. For each $P_\ell \in C_i^1$, P_ℓ randomly samples s_ℓ and distributes $[s_\ell]_t$ to C_i^2 .
2. All parties in C_i^2 can locally compute

$$([r_1]_t, \dots, [r_{t+1}]_t)^T = M \cdot ([s_1]_t, \dots, [s_n]_t)^T$$

Then they share $[s_\ell]_t$ independently with the existing sharing $[s_\ell]_t^{C_i^2}$ to C_{i+1}^1 (by randomly sampling different coefficients and discarding duplicate sharing).

3. All parties in C_{i+1}^1 can locally compute

$$([r_1]_t, \dots, [r_{t+1}]_t)^T = M \cdot ([s_1]_t, \dots, [s_n]_t)^T$$

Fig. 18: **Protocol for generating same rand sharings**

4.3 Fluid-Main Protocol

In the last section, we present the main protocol $\Pi_{\text{fluid-semimain}}$ in the fluid model against semi-honest adversaries. Specifically, the input and output operations can be easily implemented through distributing and reconstructing sharings in the semi-honest setting, while a complete illustration is included in Figure 20.

Adapting to *submaximal fluidity*. In the execution phase of $\Pi_{\text{fluid-semimain}}$, we replace C_i with C_i^2 and modify the final step to "Set $i = i + 1$ ".

Theorem 8. *Protocol $\Pi_{\text{fluid-semimain}}$ shown in Figure 20 securely realizes Functionality $\mathcal{F}_{\text{dabb}}$ shown in Figure 1 in the $(\mathcal{F}_{\text{semi-mult}}, \mathcal{F}_{\text{semi-trans}})$ -hybrid model with information-theoretic security in the presence of a semi-honest adversary controlling t corrupted parties in each committee.*

Proof. A formal proof refers to Appendix F.

5 Achieving Malicious Security

In the last section, we construct a verification phase to guarantee malicious security in the honest majority setting under *maximal fluidity* and *submaximal fluidity*. For the *submaximal fluidity* setting, we observe that both settings share the same round complexity for multiplication and transfer. Consequently, we adapt the *maximal fluidity* verification protocol to the *submaximal fluidity* setting by simply treating distinct rounds within the same committee as interactions

Protocol $\Pi_{\text{lon-2}}$

Inputs: Two adjacent committees C_i and C_{i+1} with a real world adversary A controlling the minority of C_i and C_{i+1} respectively. Let N_{2i-1} and N_{2i} be the number of multiplication gates in the $(2i-1)$ -th layer and $(2i)$ -th layer (Assuming that $N_{2i-1}, N_{2i} \geq n$)

Protocols:

1. C_i and C_{i+1} call $\mathcal{F}_{\text{split-samerand}}$ N_{2i-1} times to obtain enough random sharings $([r_{i,\ell}]_t)_{i \in [n], \ell \in [\frac{N_{2i-1}}{n}]}$.
2. C_i and C_{i+1} invoke $\Pi_{\text{lon-1}}$ on N_{2i} pairs of random sharings evaluated in the $(2i)$ th layer.
3. C_i calls $\mathcal{F}_{\text{zero}}$ N_{2i-1} times to obtain enough zero sharings.
4. Firstly, C_i^2 invokes $\Pi_{\text{expand-zero}}$ on zero sharings generated by $\mathcal{F}_{\text{zero}}$ to obtain t -wise Independence sharings $([\hat{0}_{1,\ell}]_{2t}, \dots, [\hat{0}_{n,\ell}]_{2t})_{\ell \in [\frac{N_{2i-1}}{n}]}$. For each round $\ell = 1, \dots, \frac{N_{2i-1}}{n}$, C_{i+1}^2 locally computes a set of n sharings $([x_{1,\ell} \cdot y_{1,\ell} + \hat{0}_{1,\ell}]_{2t} + [r_{1,\ell}]_t, \dots, [x_{n,\ell} \cdot y_{n,\ell} + \hat{0}_{n,\ell}]_{2t} + [r_{n,\ell}]_t)$, and sends $[x_{j,\ell} \cdot y_{j,\ell} + \hat{0}_{j,\ell}]_{2t} + [r_{j,\ell}]_t$ to $P_j \in C_{i+1}^1$.
5. For each round $\ell = 1, \dots, \frac{N_{2i-1}}{n}$, all parties in C_{i+1}^1 behaves like a P_{king} sequentially. Each $P_j \in C_{i+1}^1$ reconstructs the secret $e_{j,\ell} = x_{j,\ell} \cdot y_{j,\ell} + r_{j,\ell}$ and sends $e_{j,\ell}$ to C_{i+1}^2 .
6. All parties in C_{i+1}^2 locally compute $[z_{i,\ell}]_t = e_{i,\ell} - [r_{i,\ell}]_t$ for $i \in [n], \ell \in [\frac{N_{2i-1}}{n}]$. And they can perform multiplications in the $(2i)$ th layer locally with the help of the outputs of $\Pi_{\text{lon-1}}$ and a set of $([z_{i,\ell}]_t = e_{i,\ell} - [r_{i,\ell}]_t)_{i \in [n], \ell \in [\frac{N_{2i-1}}{n}]}$.

Fig. 19: lon-2 Multiplication

between different committees. For simplicity, we omit the formal description of the *submaximal fluidity* verification protocol.

According to [9], Choudhuri et al. extend the definition of linear-based MPC and weak privacy introduced from [19] to the fluid model, and also prove that any linear-based and weak private fluid MPC with a minority corruption of the clients and each committee is secure against a malicious R -adaptive adversary up to additive attacks. Consequently, our fluid-DN protocols are secure up to the additive attack in the presence of a malicious R -adaptive adversary.

To guarantee the correctness of the whole computation, we need to check the following:

- Each P_{king} sends the consistent values to all parties in the next committee.
- All multiplications and transfers are correctly computed.

For the first part, verification is straightforward by computing a random linear combination of the received values. Details are presented in Figure 21. For the remaining part, following [9, 26, 4], we use an incremental check to verify

Protocol $\Pi_{\text{fluid-semimain}}$

Input Phase: Each client $U_i \in C_{\text{clnt}}$ holds an input x_i and distributes $[x_i]_t$ to the first committee C_1 .

Execution Phase:

1. Addition Gate : C_i holds two sharings $[x]_t^{C_i}$ and $[y]_t^{C_i}$. And C_i locally computes $[x + y]_t^{C_i}$ and invokes $\mathcal{F}_{\text{semi-trans}}$ if needed.
2. Multiplication Gate : C_i holds two sharings $[x]_t^{C_i}$ and $[y]_t^{C_i}$. C_i invokes $\mathcal{F}_{\text{semi-mult}}$ on two sharings.
3. Relay Gate : C_i holds a sharing $[x]_t^{C_i}$. C_i invokes $\mathcal{F}_{\text{semi-trans}}$ if needed.
4. Set $i = i + 2$, C_i continues to evaluate the gates in the next layer.

Output phase: The final committee C_ℓ computes sharings of final outputs $[z]_t$. C_ℓ returns outputs to the corresponding client.

Fig. 20: Main protocol for maximal-fluid MPC

Protocol $\Pi_{\text{publicvalue-check}}$

Inputs: Denote $e_1, e_2, \dots, e_p$ to be the values held by C_{i+2} .

Protocols:

1. C_{i+2} invokes $\mathcal{F}_{\text{coin}}$ to obtain a random r .
2. For each $P_j \in C_{i+2}$, P_j calculates $u_j = \sum_{u=1}^p r^{u-1} \cdot e_u$ and sends u_j to all parties in C_{i+3} .
3. Each $P_j \in C_{i+3}$ checks whether it holds n same values. If not, it sends **abort** to all parties in C_{i+4} .

Fig. 21: Protocol for checking public values

the correctness of all multiplication gates and all transfers. Indeed, there exists a global random sharing $[r]_t$ to authenticate each output sharing $[z]_t$ to each wire, computed as $[r \cdot z]_t$. Then, a committee C_i first receives two accumulated check sharings from the previous committee, updates the accumulated check sharings after randomizing all paired input sharings ($[z]_t, [r \cdot z]_t$) held by C_i , and passes updated check sharings to the following committee. In the end, the last committee executes the final check.

The incremental check for each committee is formalized as $\Pi_{\text{incre-check}}$ shown in Figure 22. Meanwhile, we also propose our fluid main protocol $\Pi_{\text{fluid-malmain}}$ to defend against malicious adversaries, which is illustrated in Figure 23.

Communication Cost of Verification. The primary overhead of the verification protocol stems from the computation of additional multiplications, i.e., $[r \cdot z]_t$ for each output sharing $[z]_t$ of all gates. Consequently, the cost of evalu-

Protocol $\Pi_{\text{inc-check}}$

Inputs: The committee C_i ($i \geq 3$) receives the accumulated sharings $[u_{i-2}]_t$ and $[v_{i-2}]_t$ together with $[r]_t$ from C_{i-2} . C_i also holds $([z_j]_t)_{j \in [p]}$ as the inputs, together with an authenticated version $([r \cdot z_j]_t)_{j \in [p]}$.

Protocols:

1. C_i invokes $\mathcal{F}_{\text{coin}}$ to obtain a random r_i .
2. C_i updates $[u_i]_t = [u_{i-2}]_t + \sum_{j=1}^p r_i^{j-1} \cdot [z_j]_t$ and $[v_i]_t = [v_{i-1}]_t + \sum_{j=1}^p r_i^{j-1} \cdot [r \cdot z_j]_t$.
3. C_i invokes Π_{trans} to transfer $[u_i]_t$, $[v_i]_t$ and $[r]_t$ to C_{i+2} .

Fig. 22: **Protocol for incremental check**

ating a single gate in the malicious setting becomes the cost of the semi-honest case plus the cost of one additional multiplication. Concretely, the amortized cost in Velora-1 and Velora-2 (resp., lon-1 and lon-2 for *submaximal fluidity*) of a multiplication gate per party increase to 11 and 13.5 (resp., 8 and 11) field elements, and of an addition or relay gate per party 8.5 and 7 (resp., 7 and 5.5) field elements in the malicious setting.

We present the following lemma to prove that the verification protocol effectively ensures the correctness of the overall computation. A formal proof of Lemma 3 can refer to Appendix G. Integrating our results in the semi-honest setting and Lemma 3, we can naturally derive Corollary 4.

Lemma 3. *Given a sufficiently large field $\mathbb{F}$, the verification phase is correct, i.e., if it accepts all multiplication gates and all transfers are computed correctly. In addition, it rejects except with negligible probability if there exists one transfer or one multiplication gate not correctly computed.*

Corollary 4. *Protocol $\Pi_{\text{fluid-malmain}}$ shown in Figure 23 securely realizes Functionality $\mathcal{F}_{\text{dabb}}$ shown in Figure 1 with information-theoretic security in the $(\mathcal{F}_{\text{rand}}, \mathcal{F}_{\text{doublerand}}, \mathcal{F}_{\text{coin}})$ -hybrid model in the presence of malicious adversaries controlling t corrupted parties in each committee.*

Remark: If $\Pi_{\text{fluid-malmain}}$ adopts the protocol $\Pi_{\text{Velora-2}}$, the hybrid model of Corollary 4 will be $(\mathcal{F}_{\text{rand}}, \mathcal{F}_{\text{doublerand}}, \mathcal{F}_{\text{coin}}, \mathcal{F}_{\text{zero}})$. For *submaximal fluidity setting*, if $\Pi_{\text{fluid-malmain}}$ adopts the protocol $\Pi_{\text{lon-1}}$, the hybrid model of Corollary 4 will be $(\mathcal{F}_{\text{rand}}, \mathcal{F}_{\text{split-doublerand}}, \mathcal{F}_{\text{coin}})$; If $\Pi_{\text{fluid-malmain}}$ adopts the protocol $\Pi_{\text{lon-2}}$, the hybrid model of Corollary 4 will be $(\mathcal{F}_{\text{rand}}, \mathcal{F}_{\text{split-samerand}}, \mathcal{F}_{\text{coin}}, \mathcal{F}_{\text{zero}})$.

Protocol $\Pi_{\text{fluid-malmain}}$

Input Phase:

1. C_1 invokes $\mathcal{F}_{\text{rand}}$ to obtain a random $[r]_t$. (Assume there exists another committee C_0 that helps C_1).
2. Each $U_j \in C_{\text{clnt}}$ shares $[x_j]_t$ to C_1 .
3. C_1 invokes Π_{trans} on $([x_j]_t)_{j \in [n]}$ and $[r]_t$. C_1 invokes $\Pi_{\text{Velora-1}}$ to compute $([rx_j]_t)_{j \in [n]}$.
4. C_3 obtains $([x_j]_t)_{j \in [n]}$, $([rx_j]_t)_{j \in [n]}$ and $[r]_t$. C_3 uses $([x_j]_t)_{j \in [n]}$ as the inputs to evaluate gates in the first layer. C_3 invokes $\Pi_{\text{incre-check}}$ to start the incremental check.

Execution Phase: For the committee C_{2i-1} ($i \geq 2$), they operate as follows:

1. C_{2i-1} performs computations as follows :
 - Addition Gate: C_{2i-1} holds two input sharings $[x]_t^{C_{2i-1}}$ and $[y]_t^{C_{2i-1}}$. C_{2i-1} locally computes $[x+y]_t^{C_{2i-1}}$ and $[rx+ry]_t^{C_{2i-1}}$. Then C_{2i-1} invokes Π_{trans} on $[x+y]_t^{C_{2i-1}}$ and $[rx+ry]_t^{C_{2i-1}}$.
 - Multiplication Gate: C_{2i-1} holds two input sharings $[x]_t^{C_{2i-1}}$ and $[y]_t^{C_{2i-1}}$. C_{2i-1} invokes $\Pi_{\text{Velora-1}}$ to compute $[x \cdot y]_t^{C_{2i+1}}$ and $[rx \cdot y]_t^{C_{2i+1}}$.
 - Relay Gate: C_{2i-1} holds a sharing $[x]_t^{C_{2i-1}}$. C_i^2 invokes Π_{trans} on $[x]_t^{C_{2i-1}}$ and $[rx]_t^{C_{2i-1}}$.
2. C_{2i-1} invokes $\Pi_{\text{incre-check}}$ to update $([u_{2i-1}]_t, [v_{2i-1}]_t)$.

Verification Phase: The last committee C_ℓ holds the last three checking sharings $([r]_t, [u_\ell]_t, [v_\ell]_t)$. C_ℓ first opens $[r]$, and sends **abort** if $[r]_t$ is inconsistent. C_ℓ continues to open $[v_\ell]_t - r \cdot [u_\ell]_t$, and sends **abort** if $[v_\ell]_t - r \cdot [u_\ell]_t$ is inconsistent. Finally, C_ℓ checks whether $v_\ell - r \cdot u_\ell = 0$. If not, C_ℓ aborts. Otherwise, C_ℓ turns to the next phase.

Output phase: If the verification passes, the last committee C_ℓ returns the output sharings to C_{clnt} .

Fig. 23: **Protocol for fluid MPC against malicious adversaries**

References

1. Beaver, D.: Efficient Multiparty Protocols Using Circuit Randomization. In: Feigenbaum, J. (ed.) Advances in Cryptology – CRYPTO’91. Lecture Notes in Computer Science, vol. 576, pp. 420–432. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 11–15, 1992). https://doi.org/10.1007/3-540-46766-1_34
2. Benhamouda, F., Halevi, S., Krawczyk, H., Miao, A., Rabin, T.: Threshold Cryptography as a Service (in the Multiserver and YOSO Models). In: Yin, H., Stavrou, A., Cremers, C., Shi, E. (eds.) ACM CCS 2022: 29th Conference on Computer

- and Communications Security. pp. 323–336. ACM Press, Los Angeles, CA, USA (Nov 7–11, 2022). <https://doi.org/10.1145/3548606.3559397>
3. Bhadauria, R., Yu Chiang, J.H., Ravi, D., Sternad, J., Yakoubov, S.: Minimizing the Use of the Honest Majority in YOSO MPC with Guaranteed Output Delivery. Cryptology ePrint Archive, Paper 2024/2059 (2024), <https://eprint.iacr.org/2024/2059>
 4. Bienstock, A., Escudero, D., Polychroniadou, A.: On Linear Communication Complexity for (Maximally) Fluid MPC. In: Handschuh, H., Lysyanskaya, A. (eds.) *Advances in Cryptology – CRYPTO 2023, Part I. Lecture Notes in Computer Science*, vol. 14081, pp. 263–294. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 20–24, 2023). https://doi.org/10.1007/978-3-031-38557-5_9
 5. Bienstock, A., Escudero, D., Polychroniadou, A.: Perfectly Secure Fluid MPC with Abort and Linear Communication Complexity. *IACR Communications in Cryptology* 1(4) (2025)
 6. Braun, L., Damgård, I., Orlandi, C.: Secure Multiparty Computation from Threshold Encryption Based on Class Groups. In: Handschuh, H., Lysyanskaya, A. (eds.) *Advances in Cryptology – CRYPTO 2023, Part I. Lecture Notes in Computer Science*, vol. 14081, pp. 613–645. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 20–24, 2023). https://doi.org/10.1007/978-3-031-38557-5_20
 7. Cascudo, I., David, B., Garms, L., Konring, A.: YOLO YOSO: Fast and Simple Encryption and Secret Sharing in the YOSO Model. In: Agrawal, S., Lin, D. (eds.) *Advances in Cryptology – ASIACRYPT 2022, Part I. Lecture Notes in Computer Science*, vol. 13791, pp. 651–680. Springer, Cham, Switzerland, Taipei, Taiwan (Dec 5–9, 2022). https://doi.org/10.1007/978-3-031-22963-3_22
 8. Chida, K., Genkin, D., Hamada, K., Ikarashi, D., Kikuchi, R., Lindell, Y., Nof, A.: Fast Large-Scale Honest-Majority MPC for Malicious Adversaries. In: Shacham, H., Boldyreva, A. (eds.) *Advances in Cryptology – CRYPTO 2018, Part III. Lecture Notes in Computer Science*, vol. 10993, pp. 34–64. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 19–23, 2018). https://doi.org/10.1007/978-3-319-96878-0_2
 9. Choudhuri, A.R., Goel, A., Green, M., Jain, A., Kaptchuk, G.: Fluid MPC: Secure Multiparty Computation with Dynamic Participants. In: Malkin, T., Peikert, C. (eds.) *Advances in Cryptology – CRYPTO 2021, Part II. Lecture Notes in Computer Science*, vol. 12826, pp. 94–123. Springer, Cham, Switzerland, Virtual Event (Aug 16–20, 2021). https://doi.org/10.1007/978-3-030-84245-1_4
 10. Damgård, I., Nielsen, J.B.: Scalable and Unconditionally Secure Multiparty Computation. In: Menezes, A. (ed.) *Advances in Cryptology – CRYPTO 2007. Lecture Notes in Computer Science*, vol. 4622, pp. 572–590. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 19–23, 2007). https://doi.org/10.1007/978-3-540-74143-5_32
 11. Damgård, I.B., Kamp, S.H., Loss, J., Nielsen, J.B.: Asynchronous YOSO a la Pailier. Cryptology ePrint Archive, Paper 2025/128 (2025), <https://eprint.iacr.org/2025/128>
 12. David, B., Deligios, G., Goel, A., Ishai, Y., Konring, A., Kushilevitz, E., Liu-Zhang, C.D., Narayanan, V.: Perfect MPC over Layered Graphs. In: Handschuh, H., Lysyanskaya, A. (eds.) *Advances in Cryptology – CRYPTO 2023, Part I. Lecture Notes in Computer Science*, vol. 14081, pp. 360–392. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 20–24, 2023). https://doi.org/10.1007/978-3-031-38557-5_12

13. David, B., Mondal, A., Satish, R.: Rumors MPC: GOD for Dynamic Committees, Low Communication via Constant-Round Chat. *Cryptology ePrint Archive*, Paper 2025/1464 (2025), <https://eprint.iacr.org/2025/1464>
14. Deligios, G., Konring, A., Liu-Zhang, C.D., Narayanan, V.: Statistical layered MPC. In: *Theory of Cryptography Conference*. pp. 362–394. Springer (2025)
15. Erwig, A., Faust, S., Riahi, S.: Large-Scale Non-Interactive Threshold Cryptosystems in the YOSO Model. *Cryptology ePrint Archive*, Paper 2021/1290 (2021), <https://eprint.iacr.org/2021/1290>
16. Escudero, D., Masserova, E., Polychroniadou, A.: Towards Scalable YOSO MPC via Packed Secret-Sharing. *Cryptology ePrint Archive*, Paper 2025/635 (2025), <https://eprint.iacr.org/2025/635>
17. Feng, D., Yang, K.: Concretely efficient secure multi-party computation protocols: survey and more. *Security and Safety* **1**, 2021001 (2022)
18. Franklin, M., Yung, M.: Communication complexity of secure computation. In: *Proceedings of the twenty-fourth annual ACM symposium on Theory of computing*. pp. 699–710 (1992)
19. Genkin, D., Ishai, Y., Prabhakaran, M.M., Sahai, A., Tromer, E.: Circuits resilient to additive attacks with applications to secure computation. In: *Proceedings of the forty-sixth annual ACM symposium on Theory of computing*. pp. 495–504 (2014)
20. Gentry, C., Halevi, S., Krawczyk, H., Magri, B., Nielsen, J.B., Rabin, T., Yacoubov, S.: YOSO: You Only Speak Once - Secure MPC with Stateless Ephemeral Roles. In: Malkin, T., Peikert, C. (eds.) *Advances in Cryptology – CRYPTO 2021, Part II. Lecture Notes in Computer Science*, vol. 12826, pp. 64–93. Springer, Cham, Switzerland, Virtual Event (Aug 16–20, 2021). https://doi.org/10.1007/978-3-030-84245-1_3
21. Goyal, V., Li, H., Ostrovsky, R., Polychroniadou, A., Song, Y.: ATLAS: Efficient and Scalable MPC in the Honest Majority Setting. In: Malkin, T., Peikert, C. (eds.) *Advances in Cryptology – CRYPTO 2021, Part II. Lecture Notes in Computer Science*, vol. 12826, pp. 244–274. Springer, Cham, Switzerland, Virtual Event (Aug 16–20, 2021). https://doi.org/10.1007/978-3-030-84245-1_9
22. Lindell, Y., Nof, A.: A Framework for Constructing Fast MPC over Arithmetic Circuits with Malicious Adversaries and an Honest-Majority. In: Thuraisingham, B.M., Evans, D., Malkin, T., Xu, D. (eds.) *ACM CCS 2017: 24th Conference on Computer and Communications Security*. pp. 259–276. ACM Press, Dallas, TX, USA (Oct 31 – Nov 2, 2017). <https://doi.org/10.1145/3133956.3133999>
23. Liu-Zhang, C.D., Masserova, E., Ribeiro, J., Soni, P., Thyagarajan, S.A.: Improved YOSO Randomness Generation with Worst-Case Corruptions. *Cryptology ePrint Archive*, Paper 2024/1268 (2024), <https://eprint.iacr.org/2024/1268>
24. Nielsen, J.B., Ribeiro, J.L., Obremski, M.: Public Randomness Extraction with Ephemeral Roles and Worst-Case Corruptions. In: Dodis, Y., Shrimpton, T. (eds.) *Advances in Cryptology – CRYPTO 2022, Part I. Lecture Notes in Computer Science*, vol. 13507, pp. 127–147. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 15–18, 2022). https://doi.org/10.1007/978-3-031-15802-5_5
25. Nordholt, P.S., Veeningen, M.: Minimising Communication in Honest-Majority MPC by Batchwise Multiplication Verification. In: Preneel, B., Vercauteren, F. (eds.) *ACNS 2018: 16th International Conference on Applied Cryptography and Network Security. Lecture Notes in Computer Science*, vol. 10892, pp. 321–339. Springer, Cham, Switzerland, Leuven, Belgium (Jul 2–4, 2018). https://doi.org/10.1007/978-3-319-93387-0_17

26. Rachuri, R., Scholl, P.: Le Mans: Dynamic and Fluid MPC for Dishonest Majority. In: Dodis, Y., Shrimpton, T. (eds.) *Advances in Cryptology – CRYPTO 2022*, Part I. Lecture Notes in Computer Science, vol. 13507, pp. 719–749. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 15–18, 2022). https://doi.org/10.1007/978-3-031-15802-5_25
27. Shamir, A.: How to Share a Secret. *Communications of the Association for Computing Machinery* **22**(11), 612–613 (Nov 1979). <https://doi.org/10.1145/359168.359176>
28. Wagh, S., Tople, S., Benhamouda, F., Kushilevitz, E., Mittal, P., Rabin, T.: Falcon: Honest-Majority Maliciously Secure Framework for Private Deep Learning. *Proceedings on Privacy Enhancing Technologies* **1**, 188–208 (2021)
29. Yao, A.C.C.: Protocols for Secure Computations (Extended Abstract). In: 23rd Annual Symposium on Foundations of Computer Science. pp. 160–164. IEEE Computer Society Press, Chicago, Illinois (Nov 3–5, 1982). <https://doi.org/10.1109/SFCS.1982.38>
30. Yao, A.C.C.: How to Generate and Exchange Secrets (Extended Abstract). In: 27th Annual Symposium on Foundations of Computer Science. pp. 162–167. IEEE Computer Society Press, Toronto, Ontario, Canada (Oct 27–29, 1986). <https://doi.org/10.1109/SFCS.1986.25>

Supplementary Material

A Corruption Model

The formal and complete definition refers to [9].

Corruption threshold: In this paper, we consider an honest majority model in the fluid setting where the adversary can corrupt any minority of the clients as well as any minority of the parties in each committee in an epoch.

Corruption Timing: For clients, we suppose that the adversary specifies the list of corrupted clients before evaluating the protocol. For each committee C_i , as shown in [9], only in the *hand-off* phase of C_{i-1} does the adversary know the identities of C_i . We allow that the adversary can adaptively corrupt parties in C_i from the *hand-off* phase of C_{i-1} to the *hand-off* phase of C_i as desired.

Retroactive effect: Assuming that the adversary corrupts a party $P_\ell \in C_i$, the adversary learns the entire past state of P_ℓ , sends and receives messages on behalf of P_ℓ in the i th epoch. Hence, we have to insist that the party P_ℓ was also corrupted in each committee C_k for $k < i$. To prevent the adversary from arbitrarily learning information about prior epochs, the adversary is limited to corrupting parties in the committee C_i if corrupting a party P and its retroactive effect of considering P to be corrupted in all prior epochs does not cross the corruption threshold in any epoch.

B Benefits of The *Client-Server* Model

In the *client-server* model, we can assume that the optimal corruption is valid in each committee. Supposing the adversary A controls only $t' < t$ parties in one

committee C_i . We can construct a new adversary A' that controls additional $t - t'$ parties and operates as follows:

1. For a party controlled by A , A' follows A .
2. For a party not corrupted by A , but controlled by A' , A' instructs it to behave as an honest party.

If a protocol is secure against A' , then this protocol is also secure against A since the additional $t - t'$ parties behave honestly in both cases. Hence, we assume that the adversary corrupts exactly t_1 parties in all committees.

C Secret Sharing

In this paper, we use the standard Shamir secret sharing scheme [27] and the packed secret sharing scheme [18] to realize the fluid MPC. Let $\beta_1, \dots, \beta_k, \alpha_1, \dots, \alpha_n \in \mathbb{F}$ be $n + k$ distinct nonzero field elements. We first assume that $|\mathbb{F}| > 2n$ and $t \geq k$. Given a secret x , a degree- d Shamir sharing $[x]_d$ can be distributed by randomly sampling a polynomial f of degree at most d with x as the constant term, then sending $f(\alpha_j)$ to the j -th party P_j . Similarly, given a secret vector $\mathbf{x} = (x_1, \dots, x_k)$, a degree- $(d + k - 1)$ packed secret sharing $[\mathbf{x}]_{d+k-1}$ can be distributed by randomly sampling a polynomial g of degree at most $(d + k - 1)$ satisfying $g(\beta_i) = x_i$ for all $i \in [k]$, then sending $g(\alpha_j)$ to the j -th party P_j .

For privacy, a degree- d Shamir sharing $[x]_d$ tolerates up to d leaked shares and a degree- $(d + k - 1)$ packed secret sharing $[\mathbf{x}]_{d+k-1}$ tolerates up to d leaked shares either. It is straightforward to observe that these two sharings hold a linear homomorphism because their underlying polynomials are linearly closed. Meanwhile, the Shamir sharing exhibits a partial multiplicative homomorphism as follows:

$$[x]_{d_1} \cdot [y]_{d_2} = [x \cdot y]_{d_1+d_2}$$

only if $d_1 + d_2 < n$.

D Useful Building Blocks

Generating random sharing. We denote $\mathcal{F}_{\text{rand}}$ to be the Functionality that distributes a random degree- t sharing $[r]_t$ to all parties in a committee C_i . We also construct a protocol Π_{rand} to securely realize $\mathcal{F}_{\text{rand}}$. Their construction follow $\mathcal{F}_{\text{doublerand}}$ and $\Pi_{\text{doublerand}}$ similarly. $\mathcal{F}_{\text{rand}}$ and Π_{rand} are illustrated in Figure 24 and Figure 25 respectively.

Adapting to *submaximal fluidity*. If C_{i+1}^1 wants to obtain a random $[r]_t$, we replace the committee pair (C_i, C_{i+1}) in Π_{rand} with (C_i^2, C_{i+1}^1) . If C_i^2 wants to obtain a random $[r]_t$, we replace the committee pair (C_i, C_{i+1}) in Π_{rand} with (C_i^1, C_i^2) .

Functionality $\mathcal{F}_{\text{rand}}$

This Functionality runs with the committee C_i and operates as follows:

1. Upon receiving the shares of $(r_v)_{\{v \in \text{Corr}_{C_i}\}}$ from the adversary, $\mathcal{F}_{\text{rand}}$ randomly samples a r and reconstructs a degree- t sharing $[r]_t$ based on r and t shares as $(r_v)_{\{v \in \text{Corr}_{C_i}\}}$.
2. Finally, $\mathcal{F}_{\text{rand}}$ sends the shares of $[r]_t$ to honest parties in C_i .

Fig. 24: Random sharings in fluid setting

Protocol Π_{rand}

Inputs: Given a $(t+1) \times n$ Vandermonde matrix M .

Outputs: C_i obtains $(t+1)$ random sharings $([r_u]_t)_{u \in [t+1]}$.

Protocols:

1. For each party $P_j \in C_{i-1}$, P_j randomly samples an s_j and distributes $[s_j]_t$ to C_i .
2. All parties in C_i locally compute as follows:

$$([r_1]_t, \dots, [r_{t+1}]_t)^T = M \cdot ([s_1]_t, \dots, [s_n]_t)^T$$

3. All parties in C_i outputs $(t+1)$ random sharings $([r_u]_t)_{u \in [t+1]}$.

Fig. 25: Protocol for generating rand sharings

Theorem 9. *Protocol Π_{rand} shown in Figure 25 securely realizes Functionality $\mathcal{F}_{\text{rand}}$ shown in Figure 24 with information-theoretic security in the presence of semi-honest adversaries controlling t corrupted parties in each committee.*

Proof. In this proof, we consider a stronger scenario, i.e., when facing malicious adversaries. We construct a simulator S to simulate the behaviors of honest parties among two adjacent committees C_{i-1} and C_i .

Simulation for Π_{rand} : In Step 1 of Π_{rand} , for an honest party $P_j \in C_{i-1}, j \in H_{C_{i-1}}$, S randomly samples t field elements as the corrupted shares of $[s_j]_t$ and passes them to the adversary. For a corrupted party $P_j \in C_{i-1}, j \in \text{Corr}_{C_{i-1}}$, S learns $(t+1)$ honest shares in $[s_j]_t$ from the adversary, then S can reconstruct the whole sharing $[s_j]_t$ and computes the shares of $[s_j]_t$ held by corrupted parties. In Step 2, S computes all corrupted shares in $([r_u]_t)_{u \in [t+1]}$ and passes them to $\mathcal{F}_{\text{rand}}$.

Hybrids argument: Now we show that S perfectly simulates the behaviors of honest parties. Consider the following hybrids.

Hybrid₀: This is the real-world execution.

Hybrid₁: In this hybrid, S changes the way of preparing a random sharing for each honest party in C_{i-1} . For each honest party $P_j \in C_{i-1}, j \in H_{C_{i-1}}$, S executes as the following:

1. S randomly samples the corrupted shares in $[s_j]_t$ and sends them to the adversary.
2. S randomly samples an element s_j as the secret. S computes $[s_j]_t$ based on s_j and t sampled corrupted shares. S distributes the shares to the rest of the honest parties in C_i .

For each corrupted party $P_j \in C_{i-1}, j \in \text{Corr}_{C_{i-1}}$, S can locally compute the shares of $[s_j]_t$ held by corrupted parties in C_i as mentioned above.

Note that this does not change the distribution of the random sharings generated by honest parties. Then we can claim that the distribution of **Hybrid₀** is identical to **Hybrid₁**.

Hybrid₂: In this hybrid, S omits the second step in **Hybrid₁** when preparing a random sharing for each honest party. After computing corrupted shares in each $[s_j]_t$, S locally computes all corrupted shares in $([r_u]_t)_{u \in [t+1]}$ and passes them to $\mathcal{F}_{\text{rand}}$.

Let M^H be the sub-matrix of M consisting of all columns of M with indices in $H_{C_{i-1}}$ and M^C similarly. Then for $d \in \{t, 2t\}$, we can rewrite as:

$$([r_1]_t, \dots, [r_{t+1}]_t)^T = M^H \cdot ([s_j]_t, j \in H_{C_{i-1}})^T + M^C \cdot ([s_j]_t, j \in \text{Corr}_{C_{i-1}})^T$$

. Note that M^H is an invertible $(t+1) \times (t+1)$ matrix by the property of Vandermonde matrices. Therefore, we can construct a bijection between $([r_1]_t, \dots, [r_{t+1}]_t)^T$ and $([s_j]_t, j \in H_{C_{i-1}})^T$ if $([s_j]_t, j \in \text{Corr}_{C_{i-1}})^T$ is given. For any random $([r_u]_t)_{u \in [t+1]}$ generated by $\mathcal{F}_{\text{rand}}$, S can compute back a set of valid sharings $([s_j]_t)_{j \in H_{C_{i-1}}}^T$ to uniquely correspond it. Then **Hybrid₁** and **Hybrid₂** present the same random distribution between two worlds.

In summary, we can conclude that **Hybrid₂** is the execution in the ideal world and the distribution of **Hybrid₂** is identical to the distribution of **Hybrid₀**, the execution in the real world. $\square$

Generating Coin Tossing. We denote $\mathcal{F}_{\text{coin}}$ as the Functionality that randomly samples a random element $r \in \mathbb{F}$ and sends it to all parties in a committee C_i , which is described in Figure 26. We claim that Π_{coin} shown in Figure 27 securely realizes $\mathcal{F}_{\text{coin}}$.

Adapting to *submaximal fluidity*. If C_{i+1}^1 wants to obtain a public random r , we replace the committee pair (C_{i-1}, C_i) in Π_{coin} with (C_i^2, C_{i+1}^1) . If C_i^2 wants to obtain a public random r , we replace the committee pair (C_{i-1}, C_i) in Π_{coin} with (C_i^1, C_i^2) .

Theorem 10. *Protocol Π_{coin} shown in Figure 27 securely realizes Functionality $\mathcal{F}_{\text{coin}}$ shown in Figure 26 in the $\mathcal{F}_{\text{rand}}$ -hybrid model with information-theoretic security in the presence of semi-honest adversaries controlling t corrupted parties in each committee.*

Functionality $\mathcal{F}_{\text{coin}}$

This Functionality runs with C_i . This Functionality operates as follows:

1. $\mathcal{F}_{\text{coin}}$ randomly samples a $r \in \mathbb{F}$.
2. $\mathcal{F}_{\text{coin}}$ sends r to the adversary.
 - If the adversary returns **continue**, $\mathcal{F}_{\text{coin}}$ sends r to the honest parties in C_i .
 - If the adversary returns **abort**, $\mathcal{F}_{\text{coin}}$ sends **abort** to the honest parties in C_{i+1} .

Fig. 26: Coin tossing in fluid MPC

Protocol Π_{coin}

Outputs: All parties in C_i hold a random $r \in \mathbb{F}$.

Protocols:

1. C_{i-1} invokes $\mathcal{F}_{\text{rand}}$ to obtain a random $[r]_t$.
2. C_{i-1} opens $[r]_t$ to C_i .
3. Each party in $P_j \in C_i$ checks whether $[r]_t$ is a consistent degree- t sharing.
 - If true, P_j takes the secret r as output.
 - If false, P_j sends **abort** to all parties in C_{i+1} .

Fig. 27: Protocol for coin tossing

Proof. In this proof, we consider a stronger scenario, i.e., when facing malicious adversaries. We construct a simulator S to simulate the behaviors of honest parties among two adjacent committees C_{i-1} and C_i .

Simulation for Π_{coin} : In the beginning, S receives a random r from $\mathcal{F}_{\text{coin}}$. When invoking $\mathcal{F}_{\text{rand}}$, S emulates $\mathcal{F}_{\text{rand}}$ and receives a set of corrupted shares $(r_v)_{\{v \in \text{Corr}_{C_{i-1}}\}}$ from the adversary. S reconstructs the whole sharing $[r]_t$ based on the secret r and t corrupted shares $(r_v)_{\{v \in \text{Corr}_{C_{i-1}}\}}$. Then S faithfully follows Π_{coin} . If an honest party in C_i sends **abort** to C_{i+1} , S sends **abort** to $\mathcal{F}_{\text{coin}}$. Otherwise, S sends **continue** to $\mathcal{F}_{\text{coin}}$.

Note that the only difference between the real-world execution and the ideal-world execution is that, in the real world, the secret value r is randomly sampled by $\mathcal{F}_{\text{rand}}$, while in the ideal world, r is received from $\mathcal{F}_{\text{coin}}$. However, r is randomly sampled in both cases. Therefore, the distributions of both executions are identical. $\square$

Generating Zero Sharing. We denote $\mathcal{F}_{\text{zero}}$ as the Functionality that generates a random zero sharing $[0]_{2t}$, which is described in 28. We use Π_{zero} shown in Figure 29 to securely realize $\mathcal{F}_{\text{zero}}$.

Functionality $\mathcal{F}_{\text{zero}}$

This Functionality runs with C_i . This Functionality operates as follows:

1. $\mathcal{F}_{\text{zero}}$ receives the corrupted shares of $r_{v \in \text{Corr}_{C_i}}$ from the adversary.
2. $\mathcal{F}_{\text{zero}}$ randomly samples t elements as the shares of $[0]_{2t}$ held by honest parties in C_i . $\mathcal{F}_{\text{zero}}$ reconstructs $[0]_{2t}$ based on t shares held by honest parties in C_i , t shares held by corrupted parties in C_i and the underlying secret 0.
3. $\mathcal{F}_{\text{zero}}$ distributes the shares of $[0]_{2t}$ to the honest parties in C_i .

Fig. 28: **Generating zero sharing in fluid MPC**

Protocol Π_{zero}

Inputs: Given a fixed $(t+1) \times n$ Vandermonde matrix M .

Outputs: C_i obtains $(t+1)$ random zero sharings $([0_{(1)}]_{2t}, \dots, [0_{(t+1)}]_{2t})$.

Protocols:

1. For each $P_j \in C_{i-1}$, P_j distributes a random degree- $2t$ $[0^j]_{2t}$ to C_i .
2. All parties in C_i can locally compute

$$([0_{(1)}]_{2t}, \dots, [0_{(t+1)}]_{2t})^T = M \cdot ([0^1]_{2t}, \dots, [0^n]_{2t})^T$$

and output $([0_{(1)}]_{2t}, \dots, [0_{(t+1)}]_{2t})$.

Fig. 29: **Protocol for generating zero sharing**

Adapting to *submaximal fluidity*. If C_{i+1}^1 wants to obtain a random $[0]_{2t}$, we replace the committee pair (C_{i-1}, C_i) in Π_{zero} with (C_i^2, C_{i+1}^1) . If C_i^2 wants to obtain a random $[0]_{2t}$, we replace the committee pair (C_{i-1}, C_i) in Π_{rand} with (C_i^1, C_i^2) .

Theorem 11. *Protocol Π_{zero} shown in Figure 29 securely realizes Functionality $\mathcal{F}_{\text{zero}}$ shown in Figure 28 with information-theoretic security in the presence of semi-honest adversaries controlling t corrupted parties in each committee.*

Proof. In this proof, we consider a stronger scenario, i.e., when facing malicious adversaries. We construct a simulator S to simulate the behaviors of honest parties among two adjacent committees C_{i-1} and C_i .

Simulation for Π_{zero} : In Step 1 of Π_{zero} , for an honest party $P_j \in C_{i-1}, j \in H_{C_{i-1}}$, S randomly samples t field elements as the corrupted shares of $[0^j]_t$ and passes them to the adversary. For a corrupted party $P_j \in C_{i-1}, j \in \text{Corr}_{C_{i-1}}$, S learns $(t+1)$ honest shares in $[0^j]_t$ from the adversary. Then S samples a random degree- $2t$ sharing based on the secret 0 and the shares held by honest

parties, and views this degree- $2t$ sharing as the one distributed by P_j . In Step 2, S computes all corrupted shares in $([0_{(u)}]_t)_{u \in [t+1]}$ and passes them to $\mathcal{F}_{\text{rand}}$.

Hybrids argument: Now we show that S perfectly simulates the behaviors of honest parties. Consider the following hybrids.

Hybrid₀: This is the real-world execution.

Hybrid₁: In this hybrid, S changes the way of preparing a random zero sharing for each honest party in C_{i-1} . For each honest party $P_j \in C_{i-1}, j \in H_{C_{i-1}}$, S executes as the following:

1. S randomly samples the corrupted shares in $[0^j]_{2t}$ and sends them to the adversary.
2. S randomly samples another t elements as the shares of $[0^j]_{2t}$ held by honest parties in C_i . S computes $[0^j]_{2t}$ based on the secret 0, t corrupted shares and t honest shares. S distributes $[0^j]_{2t}$ to the honest parties in C_i .

For each corrupted party $P_j \in C_{i-1}, j \in \text{Corr}_{C_{i-1}}$, S can locally compute the shares of $[0^j]_{2t}$ held by corrupted parties in C_i as mentioned above.

Note that this does not change the distribution of the random zero sharings generated by honest parties. Then we can claim that the distribution of **Hybrid₀** is identical to **Hybrid₁**.

Hybrid₂: In this hybrid, S omits the second step in **Hybrid₁** when preparing a random zero sharing for each honest party. After computing corrupted shares in each $[0^j]_{2t}$, S locally computes all corrupted shares in $([0_{(u)}]_t)_{u \in [t+1]}$ and passes them to $\mathcal{F}_{\text{rand}}$.

Let M^H be the sub-matrix of M consisting of all columns of M with indices in $H_{C_{i-1}}$ and M^C similarly. Then for $d \in \{t, 2t\}$, we can rewrite as:

$$([0_{(1)}]_{2t}, \dots, [0_{(t+1)}]_{2t})^T = M^H \cdot ([0^j]_{2t}, j \in H_{C_{i-1}})^T + M^C \cdot ([0^j]_{2t}, j \in \text{Corr}_{C_{i-1}})^T$$

. Note that M^H is an invertible $(t+1) \times (t+1)$ matrix by the property of Vandermonde matrices. Therefore, we can construct a bijection between $([0_{(1)}]_{2t}, \dots, [0_{(t+1)}]_{2t})^T$ and $([0^j]_{2t}, j \in H_{C_{i-1}})^T$ if $([0^j]_{2t}, j \in \text{Corr}_{C_{i-1}})^T$ is given. For any random $([0_{(u)}]_t)_{u \in [t+1]}$ generated by $\mathcal{F}_{\text{rand}}$, S can compute back a set of valid sharings $([0^j]_t)_{j \in H_{C_{i-1}}}^T$ to uniquely correspond it. Then **Hybrid₁** and **Hybrid₂** present the same random distribution between two worlds.

In summary, we can conclude that **Hybrid₂** is the execution in the ideal world and the distribution of **Hybrid₂** is identical to the distribution of **Hybrid₀**, the execution in the real world. $\square$

Fluid Multiplication. We denote $\mathcal{F}_{\text{semi-mult}}$ as the multiplicative operation on $[x]_t$ and $[y]_t$ held by one committee C_i , which is illustrated in Figure 30.

Adapting to *submaximal fluidity*. We replace the committee pair (C_i, C_{i+2}) in $\mathcal{F}_{\text{semi-mult}}$ with (C_i^2, C_{i+1}^2) in the *submaximal fluidity* setting.

Functionality $\mathcal{F}_{\text{semi-mult}}$

This Functionality runs with two adjacent committees C_i and C_{i+2} . C_i holds the input sharings $[x]_t$ and $[y]_t$. This Functionality operates as follows:

1. Upon receiving the shares of $(x_j, y_j)_{j \in H_{C_i}}$ from honest parties in C_i , $\mathcal{F}_{\text{semi-mult}}$ reconstructs the secrets x and y and computes the shares of $[x]_t$ and $[y]_t$ held by corrupted parties in C_i . $\mathcal{F}_{\text{semi-mult}}$ sends these corrupted shares to the adversary.
2. $\mathcal{F}_{\text{semi-mult}}$ receives a set of $(z_j)_{\{j \in \text{Corr}_{C_{i+2}}\}}$ from the adversary.
3. $\mathcal{F}_{\text{semi-mult}}$ first computes $z = x \cdot y$ and reconstructs $[z]_t$ based on the secret z and t corrupted shares $(z_j)_{\{j \in \text{Corr}_{C_{i+2}}\}}$. Finally, $\mathcal{F}_{\text{semi-mult}}$ distributes $[z]_t$ to the honest parties in C_{i+2} .

Fig. 30: Mult in fluid setting

E Missing Proofs for Baseline Multiplications

In this Appendix, we present the formal proofs of the theorems that were omitted from the main body.

Theorem 12 (Theorem 1, restated). *Protocol Π_{trans} shown in Figure 3 securely realizes Functionality $\mathcal{F}_{\text{semi-trans}}$ shown in Figure 2 with information-theoretic security in the presence of a semi-honest adversary controlling t corrupted parties in each committee.*

Proof. We construct a simulator S to simulate the behaviors of honest parties among three committees C_i , C_{i+1} , and C_{i+2} . The simulation operates as follows:

1. In Step 1, S receives all $(\mathbf{x}^j)_{j \in \text{Corr}_{C_i}}$ from the adversary. S faithfully distributes $\llbracket \mathbf{x}^j \rrbracket_{2t}$ to C_{i+1} and sends honest shares of it to the adversary.
2. In Step 2, for each party $P_j \in C_i$ with $j \in [t+1] \cap H_{C_i}$ and each party $P_u \in C_{i+1}$, $u \in \text{Corr}_{C_{i+1}}$, S randomly samples a field element $x_{j,u}$ as the share of $\llbracket \mathbf{x}^j \rrbracket_{2t}$ held by P_u . Then, S can compute the corrupted shares of $\llbracket \mathbf{x} \rrbracket_{2t}$ by following the protocol, denoted as $(\hat{x}^u)_{u \in \text{Corr}_{C_{i+1}}}$.
3. In Step 3, for each $u \in \text{Corr}_{C_{i+1}}$, S faithfully distributes $[\hat{x}^u]_t$ to C_{i+2} and sends shares held by honest parties to the adversary.
4. In Step 4, for each party $P_u \in C_{i+1}$, $u \in H_{C_{i+1}}$ and each party $P_v \in C_{i+1}$, $v \in \text{Corr}_{C_{i+2}}$, S randomly samples a field element $x_{u,v}$ as the share of $[\hat{x}^u]_t$ held by P_v . In the end, S computes the corrupted shares in each $[x_p]_t$, $p \in [t+1]$ by following the protocol. S send these shares to $\mathcal{F}_{\text{semi-trans}}$.

Now we show that the real world and ideal world are identically distributed to the adversary. In Step 1 of Π_{trans} , the adversary receives t shares of $\llbracket \mathbf{x}^j \rrbracket_{2t}$ distributed by an honest $P_j \in C_i$, $j \in H_{C_i}$. Then, the shares that the adversary holds are uniformly random due to the degree- $2t$ threshold. Indeed, S samples

random field elements as these shares and sends them to the adversary. Hence, the view of the adversary is identical in the real and ideal worlds for this step. The similar argument also holds for Step 3 of Π_{trans} that an honest $P_u \in C_{i+1}, u \in H_{C_{i+1}}$ distributing $[\hat{x}^u]_t$ to C_{i+2} . As for the output, note that each $[x_p]_t$ is fully determined by the secret x_p and the shares held by corrupted parties, which are identical in both worlds. Thus, the output of honest parties is identical in both worlds, which completes the whole proof. $\square$

Theorem 13 (Theorem 2, restated). *Protocol $\Pi_{\text{doublerand}}$ shown in Figure 5 securely realizes Functionality $\mathcal{F}_{\text{doublerand}}$ shown in Figure 4 with information-theoretic security in the presence of a semi-honest adversary controlling t corrupted parties in each committee.*

Proof. In this proof, we consider a stronger scenario, i.e., when facing malicious adversaries. We construct a simulator S to simulate the behaviors of honest parties among two adjacent committees C_{i-1} and C_i .

Simulation for $\Pi_{\text{doublerand}}$: In Step 1 of $\Pi_{\text{doublerand}}$, for an honest party $P_j \in C_{i-1}, j \in H_{C_{i-1}}$, S randomly samples t field elements as the corrupted shares of $[s_j]_t$ and another t field elements as the corrupted shares of $[s_j]_{2t}$. For a corrupted party $P_j \in C_{i-1}, j \in \text{Corr}_{C_{i-1}}$, S learns $(t+1)$ honest shares in $[s_j]_t$ from the adversary, then S can reconstruct the whole sharing $[s_j]_t$. Moreover, S samples another $(t-1)$ field elements as the shares of $[s_j]_{2t}$ and reconstructs $[s_j]_{2t}$ based on the secret s_j , $(t-1)$ sampled shares, and $(t+1)$ honest shares sent by the adversary. S views $[s_j]_{2t}$ as the one distributed by P_j . In Step 2, S computes all corrupted shares in $([r_u]_t, [r_u]_{2t})_{u \in [t+1]}$ and passes them to $\mathcal{F}_{\text{doublerand}}$.

Hybrids argument: Now we show that S perfectly simulates the behaviors of honest parties. Consider the following hybrids.

Hybrid₀: This is the real-world execution.

Hybrid₁: In this hybrid, S changes the way of preparing a pair of random sharings for each honest party in C_{i-1} . For each honest party $P_j \in C_{i-1}, j \in H_{C_{i-1}}$, S executes as the following:

1. S randomly samples the corrupted shares in $([s_j]_t, [s_j]_{2t})$ and sends them to corrupted parties in C_i .
2. S randomly samples an element s_j as the secret. S computes $[s_j]_t$ based on s_j and t sampled corrupted shares. Then S continues to randomly sample t elements together with s_j and t shares of $[s_j]_{2t}$ held by corrupted parties and computes $[s_j]_{2t}$ based on these $(2t+1)$ elements.

For each corrupted party $P_j \in C_{i-1}, j \in \text{Corr}_{C_{i-1}}$, S can locally compute the shares of $([s_j]_t, [s_j]_{2t})$ held by corrupted parties in C_i as mentioned above.

Note that this does not change the distribution of the random double sharings generated by honest parties. Then we can claim that the distribution of **Hybrid₀** is identical to **Hybrid₁**.

Hybrid₂: In this hybrid, S omits the second step in **Hybrid₁** when preparing a pair of random sharings for each honest party. After computing corrupted shares in each $([s_j]_t, [s_j]_{2t})$, S locally computes all corrupted shares in $([r_u]_t, [r_u]_{2t})_{u \in [t+1]}$ and passes them to $\mathcal{F}_{\text{doublerand}}$.

Let M^H be the sub-matrix of M consisting of all columns of M with indices in $H_{C_{i-1}}$ and M^C similarly. Then for $d \in \{t, 2t\}$, we can rewrite as :

$$([r_1]_d, \dots, [r_{t+1}]_d)^T = M^H \cdot ([s_j]_d, j \in H_{C_{i-1}})^T + M^C \cdot ([s_j]_d, j \in \text{Corr}_{C_{i-1}})^T$$

for $d \in \{t, 2t\}$. Note that M^H is an invertible $(t+1) \times (t+1)$ matrix by the property of Vandermonde matrices. Therefore, we can construct a bijection between $([r_1]_d, \dots, [r_{t+1}]_d)^T$ and $([s_j]_d, j \in H_{C_{i-1}})^T$ if $([s_j]_d, j \in \text{Corr}_{C_{i-1}})^T$ is given. For any random $([r_u]_t, [r_u]_{2t})_{u \in [t+1]}$ generated by $\mathcal{F}_{\text{doublerand}}$, S can compute back a set of valid double sharings $([s_j]_t, [s_j]_{2t})_{j \in H_{C_{i-1}}}^T$ to uniquely correspond it. Then **Hybrid**₁ and **Hybrid**₂ present the same random distribution between two worlds.

In summary, we can conclude that **Hybrid**₂ is the execution in the ideal world and the distribution of **Hybrid**₂ is identical to the distribution of **Hybrid**₀, the execution in the real world. $\square$

Theorem 14 (Theorem 3, restated). *Protocol $\Pi_{\text{dn-mult}}$ shown in Figure 6 securely realizes Functionality $\mathcal{F}_{\text{semi-mult}}$ shown in Figure 30 in the $(\mathcal{F}_{\text{doublerand}}, \mathcal{F}_{\text{semi-trans}})$ -hybrid with information-theoretic security in the presence of a semi-honest adversary controlling t corrupted parties in each committee.*

Proof. We construct a simulator S to simulate the behaviors of honest parties among three committees C_i , C_{i+1} , and C_{i+2} .

We first prove that the honest shares of $[r]_{2t}$ outputted by $\mathcal{F}_{\text{doublerand}}$ are uniformly random and independent of the shares chosen by the adversary. Note that $\mathcal{F}_{\text{doublerand}}$ generates a pair of random $([r]_t, [r]_{2t})$ based on the corrupted shares $(r_u)_{\{u \in \text{Corr}_{C_i}\}}$ and $(r_v)_{\{v \in \text{Corr}_{C_i}\}}$ from the adversary. We can change the sampling process as follows:

1. $\mathcal{F}_{\text{doublerand}}$ first randomly samples $(t+1)$ field elements as the honest shares of $[r]_{2t}$. Then, $\mathcal{F}_{\text{doublerand}}$ reconstructs the whole sharing $[r]_{2t}$ and the secret r based on $(t+1)$ honest shares and t corrupted shares.
2. $\mathcal{F}_{\text{doublerand}}$ reconstructs the whole sharing $[r]_t$ based on the secret r and t corrupted shares.

It is convenient to check that the sampled pair of double sharing has the same distribution as the output double sharing from $\mathcal{F}_{\text{doublerand}}$. However, the shares of $[r]_{2t}$ are randomly sampled in the first step and are independent of the corrupted shares $(r_u)_{\{u \in \text{Corr}_{C_i}\}}$ and $(r_v)_{\{v \in \text{Corr}_{C_i}\}}$ from the adversary.

Now we start to present the simulation of S .

Simulation for $\Pi_{\text{dn-mult}}$: In Step 1, S emulates $\mathcal{F}_{\text{doublerand}}$ and receives the shares of random double sharings held by corrupted parties.

In Step 2, S receives the shares of $[x]_t$ and $[y]_t$ held by corrupted parties from $\mathcal{F}_{\text{semi-mult}}$. S computes the corrupted shares of $[e]_{2t} = [x]_t \cdot [y]_t + [r]_{2t}$.

In Step 3, S emulates $\mathcal{F}_{\text{semi-trans}}$ and receives the corrupted shares of $[r]_t$ from the adversary.

In Step 4, S samples a random field element as the share of $[e]_{2t}$ for each honest party in C_i . S faithfully follows Step 4 in $\Pi_{\text{dn-mult}}$.

In Step 5, S receives the corrupted shares of $[r]_t$ when emulating $\mathcal{F}_{\text{semi-trans}}$. In the end, S computes the shares of $[z]_t = e - [r]_t$ held by corrupted parties and passes them to $\mathcal{F}_{\text{semi-mult}}$.

It is clear that S perfectly simulates the behaviors of honest parties. As for the output, $[z]_t$ is fully determined by the secret z and the shares held by corrupted parties, which are identical in both worlds. Therefore, the output of honest parties is identical in both worlds. $\square$

Theorem 15 (Theorem 4, restated). *Protocol $\Pi_{\text{split-doubling}}$ shown in Figure 7 securely realizes Functionality $\mathcal{F}_{\text{split-doubling}}$ shown in Figure 8 with information-theoretic security in the presence of a semi-honest adversary controlling t corrupted parties in each committee.*

Proof. In this proof, we consider a stronger scenario, i.e., when facing malicious adversaries. We construct a simulator S to simulate the behaviors of honest parties among two adjacent committees C_i and C_{i+1} .

Simulation for $\Pi_{\text{split-doubling}}$: We construct a hybrid model proof in two steps. In the first step, S simulates how each party $P_\ell \in C_i$ distributes its random sharings $[s_\ell]_{2t}^{C_i^2}$ and $[s_\ell]_t^{C_{i+1}}$. For an honest party $P_\ell \in C_i$, S randomly samples t elements as the shares of $[s_\ell]_{2t}^{C_i^2}$ held by Corr_{C_i} and t elements as the shares of $[s_\ell]_t^{C_{i+1}}$ held by $\text{Corr}_{C_{i+1}}$, then sends them to A . For a corrupted party $P_\ell \in C_i$, Then S can recover s_ℓ based on the shares of $[s_\ell]_t^{C_{i+1}}$ held by honest parties, then determines the whole sharing $[s_\ell]_t^{C_{i+1}}$. S randomly sample t elements together with s_ℓ and shares of $[s_\ell]_{2t}^{C_i^2}$ held by honest parties, and views these $(2t + 1)$ elements as the random sharing distributed by P_ℓ .

In the second step, S simply transfer all shares of each pair $([r]_t^{C_{i+1}}, [r]_{2t}^{C_i^2})$ held by corrupted parties among two committees to $\mathcal{F}_{\text{split-doubling}}$.

Hybrids argument:

Hybrid₀: This is the real-world execution.

Hybrid₁: In this world, S simulates double sharings on behalf of the honest parties in C_i . For each honest party $P_\ell \in C_i$, S executes as the following:

1. S randomly samples t elements as the shares of $[s_\ell]_{2t}^{C_i^2}$ held by Corr_{C_i} and t elements as the shares of $[s_\ell]_t^{C_{i+1}}$ held by $\text{Corr}_{C_{i+1}}$ and sends $2t$ elements to Corr_{C_i} and $\text{Corr}_{C_{i+1}}$ respectively.
2. S randomly samples an element s_ℓ as the secret. S computes $[s_\ell]_t^{C_{i+1}}$ based on s_ℓ and t shares held by $\text{Corr}_{C_{i+1}}$. Then S continues to randomly sample t elements together with s_ℓ and t shares of $[s_\ell]_{2t}^{C_i^2}$ held by corrupted parties and computes $[s_\ell]_{2t}^{C_i^2}$ based on these $(2t + 1)$ elements.

For each corrupted party $P_\ell \in C_i$, S can locally compute double sharings as mentioned above.

For each honest party $P_\ell \in C_i$, both $[s_\ell]_t^{C_{i+1}}$ and $[s_\ell]_{2t}^{C_i^2}$ are uniformly random to A or corrupted parties. Thus, S does not disrupt the random distributions

generated by honest parties in **Hybrid**₀. Now we can obtain that the distribution of **Hybrid**₀ is identical to **Hybrid**₁.

Hybrid₂: In this hybrid, S simply transfer all shares of each pair $([r]_t^{C_{i+1}}, [r]_{2t}^{C_i^2})$ held by corrupted parties among two committees to $\mathcal{F}_{\text{split-doulerand}}$.

Back to the sketch proof of Theorem 2, we can quote that let $M^{H_{C_i}}$ be the sub-matrix of M consisting of all columns of M with indices in H_{C_i} and $M^{Corr_{C_i}}$ similarly. Then for $d \in \{t, 2t\}$, we can rewrite as :

$$([r_1]_d, \dots, [r_{t+1}]_d)^T = M^{H_{C_i}} \cdot ([s_\ell]_d, \ell \in H_{C_i})^T + M^{Corr_{C_i}} \cdot ([s_\ell]_d, \ell \in Corr_{C_i})^T$$

for $d \in \{t, 2t\}$.

And we have proved that there is an bijective between $([r_1]_d, \dots, [r_{t+1}]_d)^T$ and $([s_\ell]_d, \ell \in H_{C_i})^T$ if the shares of corrupted parties $([s_\ell]_d, \ell \in Corr_{C_i})^T$ are given. Hence, for every $(t+1)$ double sharings $([r_1]_d, \dots, [r_{t+1}]_d)^T$ generated by $\mathcal{F}_{\text{split-doulerand}}$, we can derive and then calculate a set of valid double sharings $([s_\ell]_d, \ell \in H_{C_i})$ held by honest parties in C_i . Then **Hybrid**₁ and **Hybrid**₂ present the same random distribution, which indicates that **Hybrid**₁ is identical to **Hybrid**₂.

In summary, we can conclude that the distribution of **Hybrid**₂ is identical to the distribution of **Hybrid**₀, where **Hybrid**₂ represents the ideal world execution and **Hybrid**₀ represents the real world execution. $\square$

Theorem 16. *Protocol $\Pi_{\text{split-mult}}$ shown in Figure 6 securely realizes Functionality $\mathcal{F}_{\text{semi-mult}}$ shown in Figure 30 (its submaximal version) in the $\mathcal{F}_{\text{split-doulerand}}$ -hybrid with information-theoretic security in the presence of a semi-honest adversary controlling t corrupted parties in each committee.*

Proof. We construct a simulator S to simulate the behaviors of honest parties among two committees C_i and C_{i+1} .

We first prove that the honest shares of $[r]_{2t}$ outputted by $\mathcal{F}_{\text{split-doulerand}}$ are uniformly random and independent of the shares chosen by the adversary. Note that $\mathcal{F}_{\text{split-doulerand}}$ generates a pair of random $([r]_t, [r]_{2t})$ based on the corrupted shares $(r_u)_{\{u \in Corr_{C_{i+1}}\}}$ and $(r_v)_{\{v \in Corr_{C_i}\}}$ from the adversary. We can change the sampling process as follows:

1. $\mathcal{F}_{\text{split-doulerand}}$ first randomly samples $(t+1)$ field elements as the honest shares of $[r]_{2t}$. Then, $\mathcal{F}_{\text{split-doulerand}}$ reconstructs the whole sharing $[r]_{2t}$ and the secret r based on $(t+1)$ honest shares and t corrupted shares.
2. $\mathcal{F}_{\text{split-doulerand}}$ reconstructs the whole sharing $[r]_t$ based on the secret r and t corrupted shares.

It is convenient to check that the sampled pair of double sharing has the same distribution as the output double sharing from $\mathcal{F}_{\text{split-doulerand}}$. However, the shares of $[r]_{2t}$ are randomly sampled in the first step and are independent of the corrupted shares $(r_u)_{\{u \in Corr_{C_{i+1}}\}}$ and $(r_v)_{\{v \in Corr_{C_i}\}}$ from the adversary.

Now we start to present the simulation of S .

Simulation for $\Pi_{\text{split-mult}}$: In Step 1, S emulates $\mathcal{F}_{\text{split-doulerand}}$ and receives the shares of random double sharings held by corrupted parties.

In Step 2, S receives the shares of $[x]_t$ and $[y]_t$ held by corrupted parties from $\mathcal{F}_{\text{semi-mult}}$. S computes the corrupted shares of $[e]_{2t} = [x]_t \cdot [y]_t + [r]_{2t}$.

In Step 3, S samples a random field element as the share of $[e]_{2t}$ for each honest party in C_i . S faithfully follows Step 3 in $\Pi_{\text{split-mult}}$.

In Step 4, S computes the shares of $[z]_t = e - [r]_t$ held by corrupted parties and passes them to $\mathcal{F}_{\text{semi-mult}}$.

It is clear that S perfectly simulates the behaviors of honest parties. As for the output, $[z]_t$ is fully determined by the secret z and the shares held by corrupted parties, which are identical in both worlds. Therefore, the output of honest parties is identical in both worlds. $\square$

F Missing Proofs for Efficient Multiplications

Lemma 4 (Lemma 1, restated). *The output double sharings in $\Pi_{\text{expand-sharings}}$ shown in Figure 10 exhibit the t -wise Independence property.*

Proof. For simplicity, we denote $H = H_{C_i}$ and $C = \text{Corr}_{C_i}$. For each $d \in \{t, 2t\}$, we can write as

$$([\hat{r}_v]_d, v \in C)^T = M_1^C \cdot ([r_1]_d, \dots, [r_t]_d)^T$$

, where M_1^C is a sub-matrix of M_1 consisting of all rows of M_1 with indices in C .

Note that M_1^C is an $t \times t$ matrix. Then, M_1^C is invertible by the property of hyper-invertibility of M_1 . There exists a one-to-one map between $([\hat{r}_v]_d, v \in C)^T$ and $([r_1]_d, \dots, [r_t]_d)^T$. Hence, the double sharings $([\hat{r}_v]_t, [\hat{r}_v]_{2t})_{v \in C}$ held by corrupted parties inherit the independence and uniform randomness of $([r_u]_t, [r_u]_{2t})_{u \in [t]}$. $\square$

Theorem 17 (Theorem 5, restated). *Protocol $\Pi_{\text{Velora-1}}$ shown in Figure 12 securely realizes Functionality $\mathcal{F}_{\text{batch-mult}}$ shown in Figure 11 in the $(\mathcal{F}_{\text{doublerand}}, \mathcal{F}_{\text{semi-trans}})$ -hybrid model with information-theoretic security in the presence of a semi-honest adversary controlling t corrupted parties in each committee.*

Proof. We construct a simulator S to simulate the behaviors of honest parties among three committees C_i , C_{i+1} , and C_{i+2} . For simplicity, we may assume that $N = n$.

We first prove that the honest shares of $[r]_{2t}$ outputted by $\mathcal{F}_{\text{doublerand}}$ are uniformly random and independent of the shares chosen by the adversary. Note that $\mathcal{F}_{\text{doublerand}}$ generates a pair of random $([r]_t, [r]_{2t})$ based on the corrupted shares $(r_u)_{\{u \in \text{Corr}_{C_i}\}}$ and $(r_v)_{\{v \in \text{Corr}_{C_i}\}}$ from the adversary. We can change the sampling process as follows:

1. $\mathcal{F}_{\text{doublerand}}$ first randomly samples $(t + 1)$ field elements as the honest shares of $[r]_{2t}$. Then, $\mathcal{F}_{\text{doublerand}}$ reconstructs the whole sharing $[r]_{2t}$ and the secret r based on $(t + 1)$ honest shares and t corrupted shares.

2. $\mathcal{F}_{\text{doublerand}}$ reconstructs the whole sharing $[r]_t$ based on the secret r and t corrupted shares.

It is convenient to check that the sampled pair of double sharing has the same distribution as the output double sharing from $\mathcal{F}_{\text{doublerand}}$. However, the shares of $[r]_{2t}$ are randomly sampled in the first step and are independent of the corrupted shares $(r_u)_{\{u \in \text{Corr}_{C_i}\}}$ and $(r_v)_{\{v \in \text{Corr}_{C_i}\}}$ from the adversary.

Furthermore, we have already proved that there exists a bijection between $([\hat{r}_v]_{2t}, v \in \text{Corr}_{C_i})^T$ and $([r_1]_{2t}, \dots, [r_t]_{2t})^T$. Thus, the shares of $[\hat{r}_v]_{2t}$ held by honest parties are uniformly random for any corrupted party $P_v, v \in \text{Corr}_{C_i}$.

Now we start to present the simulation of S .

Simulation for $\Pi_{\text{Velora-1}}$: In Step 1, S emulates $\mathcal{F}_{\text{doublerand}}$ and receives the shares of random double sharings held by corrupted parties. Then, S follows $\Pi_{\text{expand-sharings}}$ to compute the corrupted shares in $([\hat{r}_v]_t, [\hat{r}_v]_{2t})_{v \in [n]}$ (Note that $N = n$ in this proof).

In Step 2, for each $j \in [n]$, S receives the shares of $[x_j]_t$ and $[y_j]_t$ held by corrupted parties from $\mathcal{F}_{\text{batch-mult}}$. S computes the corrupted shares of $[e_j]_{2t} = [x_j]_t \cdot [y_j]_t + [\hat{r}_j]_{2t}$.

In Step 3, for each $p \in [t]$, S emulates $\mathcal{F}_{\text{semi-trans}}$ and receives the corrupted shares of $[r_p]_t$ from the adversary.

In Step 4, for each $j \in [n]$, S samples a random field element as the share of $[e_j]_{2t}$ for each honest party in C_i . S faithfully follows Step 4 in $\Pi_{\text{Velora-1}}$.

In Step 5, for each $j \in [n]$, S has learned the honest shares in $[e_j]_t$ at the end of Step 4. Then, S can compute the shares of $[e_j]_t$ held by corrupted parties based on $(t+1)$ honest shares. Also, S receives the corrupted shares of $[r_p]_t$ when emulating $\mathcal{F}_{\text{semi-trans}}$, and then invokes $\Pi_{\text{expand-sharings}}$ to compute the corrupted shares of $[\hat{r}_j]_t$. In the end, S computes the shares of $[z_j]_t = [e_j]_t - [\hat{r}_j]_t$ held by corrupted parties and passes them to $\mathcal{F}_{\text{batch-mult}}$.

It is clear that S perfectly simulates the behaviors of honest parties. As for the output, each $[z_j]_t, j \in [n]$ is fully determined by the secret z_j and the shares held by corrupted parties, which are identical in both worlds. Therefore, the output of honest parties is identical in both worlds. $\square$

Theorem 18 (Theorem 6, restated). *Protocol $\Pi_{\text{lon-1}}$ shown in Figure 13 securely realizes Functionality $\mathcal{F}_{\text{batch-mult}}$ shown in Figure 11 (its submaximal version) in the $\mathcal{F}_{\text{split-doublerand-hybrid}}$ model with information-theoretic security in the presence of a semi-honest adversary controlling t corrupted parties in each committee.*

Proof. We construct a simulator S to simulate the behaviors of honest parties among two committees C_i and C_{i+1} . For simplicity, we may assume that $N = n$.

We first prove that the honest shares of $[r]_{2t}$ outputted by $\mathcal{F}_{\text{split-doublerand}}$ are uniformly random and independent of the shares chosen by the adversary. Note that $\mathcal{F}_{\text{split-doublerand}}$ generates a pair of random $([r]_t, [r]_{2t})$ based on the corrupted shares $(r_u)_{\{u \in \text{Corr}_{C_{i+1}}\}}$ and $(r_v)_{\{v \in \text{Corr}_{C_i}\}}$ from the adversary. We can change the sampling process as follows:

1. $\mathcal{F}_{\text{split-doubling}}$ first randomly samples $(t + 1)$ field elements as the honest shares of $[r]_{2t}$. Then, $\mathcal{F}_{\text{split-doubling}}$ reconstructs the whole sharing $[r]_{2t}$ and the secret r based on $(t + 1)$ honest shares and t corrupted shares.
2. $\mathcal{F}_{\text{split-doubling}}$ reconstructs the whole sharing $[r]_t$ based on the secret r and t corrupted shares.

It is convenient to check that the sampled pair of double sharing has the same distribution as the output double sharing from $\mathcal{F}_{\text{split-doubling}}$. However, the shares of $[r]_{2t}$ are randomly sampled in the first step and are independent of the corrupted shares $(r_u)_{\{u \in \text{Corr}_{C_{i+1}}\}}$ and $(r_v)_{\{v \in \text{Corr}_{C_i}\}}$ from the adversary.

Furthermore, we have already proved that there exists a bijection between $([\hat{r}_v]_{2t}, v \in \text{Corr}_{C_i})^T$ and $([r_1]_{2t}, \dots, [r_t]_{2t})^T$. Thus, the shares of $[\hat{r}_v]_{2t}$ held by honest parties are uniformly random for any corrupted party $P_v, v \in \text{Corr}_{C_i}$.

Now we start to present the simulation of S .

Simulation for $\Pi_{\text{lon-1}}$: In Step 1, S emulates $\mathcal{F}_{\text{split-doubling}}$ and receives the shares of random double sharings held by corrupted parties.

In Step 2, S first follows $\Pi_{\text{expand-sharings}}$ to compute the corrupted shares in $([\hat{r}_v]_{2t})_{v \in [n]}$ (Note that $N = n$ in this proof). For each $j \in [n]$, S receives the shares of $[x_j]_t$ and $[y_j]_t$ held by corrupted parties from $\mathcal{F}_{\text{batch-mult}}$. S computes the corrupted shares of $[e_j]_{2t} = [x_j]_t \cdot [y_j]_t + [\hat{r}_j]_{2t}$.

In Step 3, S first follows $\Pi_{\text{expand-sharings}}$ to compute the corrupted shares in $([\hat{r}_v]_{2t})_{v \in [n]}$. For each $j \in [n]$, S samples a random field element as the share of $[e_j]_{2t}$ for each honest party in C_i . S faithfully follows Step 3 in $\Pi_{\text{lon-1}}$.

In Step 4, for each $j \in [n]$, S has learned the honest shares in $[e_j]_t$ at the end of Step 3. Then, S can compute the shares of $[e_j]_t$ held by corrupted parties based on $(t + 1)$ honest shares. In the end, S computes the shares of $[z_j]_t = [e_j]_t - [\hat{r}_j]_t$ held by corrupted parties and passes them to $\mathcal{F}_{\text{batch-mult}}$.

It is clear that S perfectly simulates the behaviors of honest parties. As for the output, each $[z_j]_t, j \in [n]$ is fully determined by the secret z_j and the shares held by corrupted parties, which are identical in both worlds. Therefore, the output of honest parties is identical in both worlds. $\square$

Lemma 5 (Lemma 2, restated). *The output zero sharings in $\Pi_{\text{expand-zero}}$ shown in Figure 14 exhibit the t -wise Independence property.*

Proof. For simplicity, we denote $H = H_{C_i}$ and $C = \text{Corr}_{C_i}$. Similar to Lemma 1, we can write as

$$([\hat{0}_v]_{2t}, v \in C)^T = M_1^C \cdot ([0_{(1)}]_{2t}, \dots, [0_{(t)}]_{2t})^T$$

, where M_1^C is a sub-matrix of M_1 consisting of all rows of M_1 with indices in C .

Note that M_1^C is an $t \times t$ matrix. Then, M_1^C is invertible by the property of hyper-invertibility of M_1 . There exists a one-to-one map between $([\hat{0}_v]_{2t}, v \in C)^T$ and $([0_{(1)}]_{2t}, \dots, [0_{(t)}]_{2t})^T$. Hence, the zero sharings $([\hat{0}_v]_{2t})_{v \in C}$ held by corrupted parties inherit the independence and uniform randomness of $([0_{(u)}]_{2t})_{u \in [t]}$. $\square$

Theorem 19 (Theorem 7, restated). *Protocol $\Pi_{\text{split-samerand}}$ shown in Figure 18 securely realizes Functionality $\mathcal{F}_{\text{split-samerand}}$ shown in Figure 17 with information-theoretic security in the presence of a semi-honest adversary controlling t corrupted parties in each committee.*

Proof. In this proof, we consider a stronger scenario, i.e., when facing malicious adversaries. We construct a simulator S to simulate the behaviors of honest parties among two adjacent committees C_{i-1} and C_i .

Simulation for $\Pi_{\text{split-samerand}}$: For an honest party $P_\ell \in C_i$, P_ℓ will distribute $[s_\ell]_t^{C_i^2}$ and $[s_\ell]_t^{C_{i+1}^1}$ on a random seed s_ℓ . S can randomly sample t elements as the shares of $[s_\ell]_t^{C_i^2}$ held by corrupted parties and another t elements similarly. For a corrupted party $P_\ell \in C_i$, S can receive the shares of $[s_\ell]_t^{C_i^2}$ and $[s_\ell]_t^{C_{i+1}^1}$ held by honest parties. Then S can recover these two secrets (which can be different) and these two whole sharings.

In the second step, S computes the shares of $[r]_t^{C_i^2}$ and $[r]_t^{C_{i+1}^1}$ held by corrupted parties and passes them to $\mathcal{F}_{\text{split-samerand}}$.

Hybrid Argument:

Hybrid₀: This is the real-world execution.

Hybrid₁: In this hybrid, S simulates how each honest party $P_\ell \in C_i$ generates two same random sharings.

- S randomly samples t elements as the shares of $[s_\ell]_t^{C_i^2}$ held by corrupted parties and continues to independently sample t elements as the shares of $[s_\ell]_t^{C_{i+1}^1}$ held by corrupted parties. S sends these $2t$ shares to the corrupted parties.
- S randomly samples a secret s_ℓ and reconstructs the whole two sharings $[s_\ell]_t^{C_i^2}$ and $[s_\ell]_t^{C_{i+1}^1}$. If these two sharings are the same, then S discards them and aborts.

If $P_\ell \in C_i$ is corrupted, then S locally computes $[s_\ell]_t^{C_i^2}$ and $[s_\ell]_t^{C_{i+1}^1}$ as mentioned above.

This simulation guarantees two independent random sharings with the same secret as an honest party does in the real world. Hence, the distribution of **Hybrid₁** is identical to **Hybrid₀**.

Hybrid₂: In this hybrid, S computes the shares of each pair $([r]_t^{C_i^2}, [r]_t^{C_{i+1}^1})$ held by corrupted parties and passes them to $\mathcal{F}_{\text{split-samerand}}$. Since the outputs of $\mathcal{F}_{\text{split-samerand}}$ can guarantee that the two sharings are independent, we can consider them separately. The remaining part seems a replica of the argument about a random degree- t sharing in Theorem 4. Thus, the distribution of **Hybrid₂** is identical to **Hybrid₁** from Theorem 4.

In summary, we can conclude that the distribution of **Hybrid₂** is identical to the distribution of **Hybrid₀**, where **Hybrid₂** represents the ideal world execution and **Hybrid₀** represents the real world execution. $\square$

Theorem 20 (Theorem 8, restated).

Protocol $\Pi_{\text{fluid-semimain}}$ shown in Figure 20 securely realizes Functionality $\mathcal{F}_{\text{dabb}}$ shown in Figure 1 in the $(\mathcal{F}_{\text{semi-mult}}, \mathcal{F}_{\text{semi-trans}})$ -hybrid model with information-theoretic security in the presence of a semi-honest adversary controlling t corrupted parties in each committee.

Proof. We construct a simulator S to simulate the behaviors of honest parties and honest clients.

Simulation for $\Pi_{\text{fluid-semimain}}$: We describe the strategy of S phase by phase.

- **Simulation for Input phase:** If $U_j \in C_{\text{clnt}}$ is honest, S randomly samples t field elements as the share of $[x_j]_t$ held by corrupted parties in C_1 and passes them to the adversary. If $U_j \in C_{\text{clnt}}$ is corrupted, S receives $(t+1)$ honest shares of $[x_j]_t$ from the adversary. S reconstructs the whole sharing $[x_j]_t$ based on $(t+1)$ honest shares and sends x_j as the input of P_j to $\mathcal{F}_{\text{dabb}}$. Note that for each input sharing, S learns the shares held by corrupted parties.
- **Simulation for Execution phase:** In the Execution phase, S simply emulates $\mathcal{F}_{\text{semi-mult}}$ and $\mathcal{F}_{\text{semi-trans}}$ during the computation. Therefore, S can compute the corrupted shares of each sharing associated with the output gates by following the protocol $\Pi_{\text{fluid-semimain}}$.
- **Simulation for Output phase:** If $U_j \in C_{\text{clnt}}$ is corrupted, S receives z_j from $\mathcal{F}_{\text{dabb}}$. S has also computed the shares of $[z_j]_t$ held by corrupted parties in the Execution phase. S reconstructs the whole sharing $[z_j]_t$ and passes the honest shares of $[z_j]_t$ to the adversary. If $U_j \in C_{\text{clnt}}$ is honest, S does not need to do anything.

Note that, during the computation, the adversary only receives messages from honest parties or in the Output phase, where the adversary receives the shares of corrupted parties and the final results. Since the corrupted shares are computed by S following the protocol, the distribution of these shares is the same in both the ideal world and the real world. The final results are also the same in both worlds according to the correctness of $\Pi_{\text{fluid-semimain}}$. Also, the output of each honest client is identical in both worlds according to the correctness of $\Pi_{\text{fluid-semimain}}$. Thus, $\Pi_{\text{fluid-semimain}}$ securely computes $\mathcal{F}_{\text{dabb}}$. $\square$

G Proof of Lemma 3

Lemma 6 (Lemma 3, restated). *Given a sufficiently large field $\mathbb{F}$, the verification phase is correct, i.e., if it accepts all multiplication gates and all transfers are computed correctly. In addition, it rejects except with negligible probability if there exists one transfer or one multiplication gate not correctly computed.*

Proof. As mentioned in Section 5, we only consider the case where the adversary injects an additive error into the output of each wire, i.e., $[z]_t \rightarrow [z + \epsilon]_t$ during the computation with $\epsilon \neq 0$.

We first assume that, if C_i performs the final check immediately, the probability that the check is deceived is p_i when the adversary injects additive

errors. Let p'_i be the probability of an event that $([u_i]_t, [v_i]_t, [r_i]_t)$ satisfying $v_i - r_i \cdot u_i = 0$, where $[r_i]_t$ denotes the true sharing of $[r]_t$ upon its transmission to C_i . Next, we consider the committee C_{i+2} . C_{i+2} will receive a batch of sharings $([z_1]_t, \dots, [z_p]_t)$ as inputs together with their authenticated sharings $([r_i z_1 + \epsilon^1]_t, \dots, [r_i z_p + \epsilon^p]_t)$. C_{i+2} also receives $([u_i + \epsilon^u]_t, [v_i + \epsilon^v]_t, [r_i + \epsilon^r]_t)$ from C_i . Now we ask C_{i+2} to perform the final check immediately. Let χ_i be the random element sampled by $\mathcal{F}_{\text{coin}}$ to C_i . There are two main cases:

1. If $([u_i]_t, [v_i]_t, [r_i]_t)$ is correct, meaning that $v_i - r_i \cdot u_i = 0$. C_{i+2} will check whether $(r_i + \epsilon^r) \cdot [(\sum_{j=1}^p \chi_{i+2}^{j-1} \cdot z_j) + (u_i + \epsilon^u)] = \sum_{j=1}^p \chi_{i+2}^{j-1} \cdot (r_i z_j + \epsilon^j) + (v_i + \epsilon^v)$, which is equivalent to $r_i \cdot \epsilon^u + \epsilon^r \cdot [(\sum_{j=1}^p \chi_{i+2}^{j-1} \cdot z_j) + (u_i + \epsilon^u)] = \sum_{j=1}^p \chi_{i+2}^{j-1} \cdot \epsilon^j + \epsilon^v$. If $\epsilon^u \neq 0$, there is only one r_i satisfying this reduced equation, leading to the probability $\frac{1}{|\mathbb{F}|}$. If $\epsilon^u = 0$, we continue to write as $\sum_{j=1}^p \chi_{i+2}^{j-1} \cdot (\epsilon^r \cdot z_j - \epsilon^j) = \epsilon^v - u_i$. If one of $\epsilon^r \cdot z_j - \epsilon^j$ is nonzero, then this equation holds for a random χ_{i+2} within the probability $\frac{p}{|\mathbb{F}|}$. If all $\epsilon^r \cdot z_j - \epsilon^j = 0$ with each z_j uniformly random to the adversary. Then all $\epsilon^r \cdot z_j - \epsilon^j = 0$ hold within the probability $\frac{1}{|\mathbb{F}|}$ if $\epsilon^j \neq 0$. Later, we further assume that $\epsilon^u = \epsilon^j = \epsilon^r = 0$, then the equation turns to $\epsilon^v - u_i = 0$, which holds within the probability $\frac{1}{|\mathbb{F}|}$ since u_i is uniformly random to the adversary. In summary, if any of $\epsilon^u, \epsilon^j, \epsilon^r, \epsilon^v, \epsilon^u$ is nonzero, the check passes within the probability $\frac{1}{|\mathbb{F}|} + (1 - \frac{1}{|\mathbb{F}|})(\frac{p}{|\mathbb{F}|} + \frac{1}{|\mathbb{F}|} + (1 - \frac{1}{|\mathbb{F}|})\frac{1}{|\mathbb{F}|}) < \frac{w+3}{|\mathbb{F}|}$.
2. If $([u_i]_t, [v_i]_t, [r_i]_t)$ is incorrect, meaning that $v_i - r_i \cdot u_i \neq 0$. Then the above check turns to $r_i \cdot (u_i + \epsilon^u) - v_i = \sum_{j=1}^p \chi_{i+2}^{j-1} \cdot (\epsilon^j - \epsilon^r \cdot z_j) + \epsilon^v - \epsilon^r \cdot (u_i + \epsilon^u)$. The event $u_i + \epsilon^u = 0$ happens with the probability $\frac{1}{|\mathbb{F}|}$. If $u_i + \epsilon^u \neq 0$, then there exists only one r_i satisfying the equation, leading to the probability $\frac{1}{|\mathbb{F}|}$. Therefore, if any of $\epsilon^u, \epsilon^j, \epsilon^r, \epsilon^v, \epsilon^u$ is nonzero, the check passes within the probability $\frac{1}{|\mathbb{F}|} + (1 - \frac{1}{|\mathbb{F}|})\frac{1}{|\mathbb{F}|} \leq \frac{2}{|\mathbb{F}|}$.

Up to this point, we can deduce that $p_i \leq p'_i \cdot \frac{2}{|\mathbb{F}|} + (1 - p'_i) \cdot \frac{w+3}{|\mathbb{F}|} < \frac{w+5}{|\mathbb{F}|}$.

Consider the committee C_3 that starts the incremental check,

- Let $[x_1]_t, \dots, [x_n]_t$ be the input sharings held by C_1 and $[x_1 + \epsilon_1]_t, \dots, [x_n + \epsilon_n]_t$ be the input sharings held by C_3 . Let $[r]_t$ be the global random sharing held by C_1 and $[r + \epsilon_r]_t$ be the transferred sharing held by C_3 . C_3 also holds n authenticated input sharings $[rx_1 + \epsilon_{r,1}]_t, \dots, [rx_n + \epsilon_{r,n}]_t$. Then, C_3 computes $[u_3]_t = \sum_{j=1}^n \chi_1^{j-1} \cdot [x_j + \epsilon_j]_t$ and $[v_3]_t = \sum_{j=1}^n \chi_1^{j-1} \cdot [rx_j + \epsilon_{r,j}]_t$. The check passes only if $(r + \epsilon_r) \cdot [\sum_{j=1}^n \chi_1^{j-1} \cdot (x_j + \epsilon_j)] = \sum_{j=1}^n \chi_1^{j-1} \cdot (rx_j + \epsilon_{r,j})$, which is equivalent to $r \cdot (\sum_{j=1}^n \chi_1^{j-1} \cdot \epsilon_j) = \sum_{j=1}^n \chi_1^{j-1} \cdot \epsilon_{r,j} - \epsilon_r \cdot [\sum_{j=1}^n \chi_1^{j-1} \cdot (x_j + \epsilon_j)]$. If any ϵ_j is nonzero, then $\sum_{j=1}^n \chi_1^{j-1} \cdot \epsilon_j \neq 0$ for a random χ_1 with at least probability $1 - \frac{n}{|\mathbb{F}|}$ according to the Schwartz-Zippel Lemma. Then a global random r satisfies the above equation with the only probability $\frac{1}{|\mathbb{F}|}$, then the check passes within the probability $\frac{n}{|\mathbb{F}|} + (1 - \frac{n}{|\mathbb{F}|}) \cdot \frac{1}{|\mathbb{F}|} < \frac{n+1}{|\mathbb{F}|}$. Now we assume that $\epsilon_j = 0$ for all j . The equation

turns to $\sum_{j=1}^n \chi_1^{j-1} \cdot (\epsilon_{r,j} - \epsilon_r \cdot x_j) = 0$. If one of $\epsilon_{r,j} - \epsilon_r \cdot x_j \neq 0$, then the new equation holds for a random χ_1 within the probability $\frac{n}{|\mathbb{F}|}$. If all $\epsilon_{r,j} - \epsilon_r \cdot x_j = 0$, then there are $(t+1)$ honest inputs that are uniformly random to the adversary. Therefore, within the probability $\frac{1}{|\mathbb{F}|}$, $\epsilon_r \neq 0$. If all $\epsilon_{r,j} - \epsilon_r \cdot x_j = 0$ and $\epsilon_r = 0$, then $\epsilon_{r,j} = 0$ for all j , meaning that no additive error in this case. In summary, if any of $\epsilon_j, \epsilon_{r,j}, \epsilon_r$ is nonzero, the check passes within the probability $\frac{n+1}{|\mathbb{F}|} + (\frac{n}{|\mathbb{F}|} + \frac{1}{|\mathbb{F}|}) = \frac{2n+2}{|\mathbb{F}|} \leq \frac{2w+2}{|\mathbb{F}|}$.

Hence, we claim that, given a sufficiently large field $\mathbb{F}$, the probability that the final check passes if there exists one transfer or one multiplication gate not correctly computed is within $\max\{\frac{2w+2}{|\mathbb{F}|}, \frac{w+5}{|\mathbb{F}|}\} < \frac{2w+5}{|\mathbb{F}|}$ and negligible, where w is the width of the whole circuit. $\square$